

Edge Detection in 3D Point Clouds for Industrial Applications

by

Linghao Du

A thesis submitted in conformity with the requirements
for the degree of Master of Applied Science
Graduate Department of Mechanical and Industrial Engineering
University of Toronto

© Copyright by Linghao Du 2020

Edge Detection in 3D Point Clouds for Industrial Applications

Linghao Du

Master of Applied Science

Department of Mechanical and Industrial Engineering
University of Toronto

2020

Abstract

3D point clouds are a commonly used medium to represent and analyze object shapes. Edge detection is important for 3D point cloud processing since it provides geometrical clues for analyzing point clouds, serving as the foundation for point cloud processing tasks and industrial applications. Existing geometry-based edge detectors use angle-dependent geometrical quantities to detect edges. Since the quantities are affected by edge angles and are similar near edges, they suffer from missed detections and redundant detections and require fine-tuned thresholds for every point clouds. This work proposes a novel edge detector to solve these problems. A new two half-planes model (THP) is proposed to estimate geometric quantities that are not affected by edge angles. We developed adapted non-maximum suppression (ANMS) by applying directional constraints on point cloud neighborhood for suppressing redundant detections. Experimental results show that the proposed detector can improve the detection result without the need to fine-tune thresholds.

Acknowledgments

Throughout the past two years of my master's study, I have received lots of support and assistance from many people I was fortunate enough to meet.

I would first like to acknowledge my supervisor, Professor Yu Sun, for offering me the opportunity to work in the Advanced Micro and Nanosystems Lab. I have learned a lot from your rigorous approach to scientific research. I am grateful for your time, support, and guidance throughout my study.

I would also like to thank all lab members of the Advanced Micro and Nanosystems Lab, for their support, expertise, and company during the past two years. I am especially grateful for the helpful discussions with Dr. Xingjian Liu and other members of the vision subgroup.

I would like to acknowledge Professor Xinyu Liu, Professor Yu Zou, and members of their labs for providing helpful discussions, insights, and support during the early stage of my study.

Additionally, I would like to thank my family for their unconditional love, support, and encouragement for my master's study. Thank you for everything they have done throughout my life. Lastly, I would like to give special thanks to my girlfriend for her support, care, and love through all these years.

Table of Contents

Acknowledgments.....	iii
Table of Contents.....	iv
List of Tables	vi
List of Figures	vii
List of Abbreviations	ix
List of Symbols and Notations.....	x
1 Introduction.....	1
1.1 Motivation.....	1
1.2 Thesis overview	4
2 Background	5
2.1 Point cloud and its sources.....	5
2.2 Edge in point clouds.....	6
2.3 Conversion-based edge detection	7
2.4 Geometry-based edge detection.....	8
2.4.1 Query points and neighborhoods	8
2.4.2 Techniques for analyzing point cloud neighborhoods	9
2.4.3 Existing geometry-based detectors	11
3 Two half-planes model to avoid missed detections	15
3.1 The problem of missed detections	15
3.2 Existing solutions for missed detections.....	15
3.3 Two half-planes model	16
4 Adapted non-maximum suppression to avoid redundant detections.....	24
4.1 The problem of redundant detections	24
4.2 Existing solutions for redundant detections.....	24
4.3 Adapted non-maximum suppression.....	25

5	Experiments and results	31
5.1	System overview	31
5.2	Performance of edge point detection using two half-planes model.....	32
5.3	Performance of adapted non-maximum suppression.....	36
5.4	Overall performance evaluation.....	41
6	Conclusions and future work	45
6.1	Conclusions.....	45
6.2	Future work.....	46
	References.....	47

List of Tables

Table 5-1. Change of precision, recall, and F-score of the detectors after direct non-maximum suppression (direct NMS) and adapted non-maximum suppression (ANMS) are applied. Averaged across all tested edge angles. Higher precision means fewer redundant detections; higher recall means fewer missed detections; and higher F-score means better overall detection performance.	39
Table 5-2. Thresholds used for the edge detectors. For the existing detectors, both fixed and tuned thresholds are used.	42
Table 5-3. Overall performance of the edge detectors. Values show the means and standard deviations. Values enclosed in parenthesis are associated with fixed thresholds.	44

List of Figures

Figure 1-1. Edges are curves along which surface directions change abruptly. Edge points are points in a point cloud that are on or sufficiently near the edges.	1
Figure 1-2. Missed detections are undetected edge points. Redundant detections are non-edge points detected as edge points.	2
Figure 1-3. Thresholds for existing geometrical quantities are dependent on edge angles [13], [17], [18]. Values in brackets are example geometrical quantities estimated by existing geometry-based detectors.	3
Figure 2-1. Curvature estimated with surface variation of point clouds with various edge angles. The color bar shows the magnitude of curvatures.	12
Figure 2-2. Possible normal clustering results. Reproduced from [18] (CC BY-NC-ND 3.0).....	13
Figure 3-1. Two half-planes model (THP). (a) the fitted two half-planes are close to the neighbor points, results in a low THP fitting error. (b) the fitted two half-planes deviate from the neighbor points, results in a high THP fitting error.	17
Figure 3-2. The process of two half-planes model (THP) fitting for potential edge point detection. Each point within the point cloud is processed by fitting two half-planes to its neighborhood. The fitting results are then used to select potential edge points for further processing.	17
Figure 3-3. Half-plane construction. (a) constructed with the query point, one sampled neighbor point, and the estimated edge direction, with half-plane boundary defined by the edge direction. (b) constructed with the query point and two sampled neighbor points, with half-plane boundary defined by the intersection line with another half-plane.	18
Figure 3-4. Half-plane distance. <i>p_{ij1}</i> and <i>p_{ij2}</i> can be projected onto the half-plane, so point-to-plane distances are used as the half-plane distance. <i>p_{ij3}</i> and <i>p_{ij4}</i> cannot be projected onto the half-plane, so point-to-line distance are used.	19
Figure 4-1. The deviation associated with a new point produced by thinning techniques.	25

Figure 4-2. The edge direction at a given location is the tangent of an edge and is perpendicular to normals in the neighborhood.	26
Figure 5-1. The system consists of a commercial structural light 3D scanner and the object to be scanned. The scanner projects structural light patterns onto the object and captures point clouds from the object.	31
Figure 5-2. Edge detection process of the proposed geometry-based edge detector.	32
Figure 5-3. Recall value comparison of edge detectors under different edge angles. Error bars show standard deviation with 5 samples. Higher recall value means fewer missed detections. ..	34
Figure 5-4. Qualitative comparison of detection results from the edge detectors. Red points and grey points represent detected edge points and non-edge points, respectively. Results with less missed detections when compared to the ground truth are better.	35
Figure 5-5. Precision, recall, and F-score comparison of edge detectors before and after applying direct non-maximum suppression (direct NMS) and adapted non-maximum suppression (ANMS). Higher precision means fewer redundant detections; higher recall means fewer missed detections; and higher F-score means better overall detection performance.	38
Figure 5-6. Qualitative comparison of detection results with direct non-maximum suppression (direct NMS) and adapted non-maximum suppression (ANMS) with the point cloud with a 90° edge. Red points and grey points represent detected edge points and non-edge points, respectively. The closer a detection result resembles the ground truth, the better it is.	40
Figure 5-7. The industrial objects and their point clouds captured from the 3D scanner.	41
Figure 5-8. F-score comparison of edge detectors applied on point clouds captured from industrial objects. Values averaged across the three point clouds. Error bars show standard deviation.	42
Figure 5-9. Qualitative comparison of detection results of the edge detectors applied on point clouds captured from industrial objects. Red points and grey points represent detected edge points and non-edge points, respectively.	43

List of Abbreviations

2D:	two-dimensional
3D:	three-dimensional
ANMS:	adapted non-maximum suppression
CAD:	computer-aided design
CMM:	coordinate measuring machine
GIS:	geographic information system
HAC:	hierarchical agglomerative clustering
HT:	Hough transform
kNN:	k nearest neighbor
NMS:	non-maximum suppression
PCA:	principal component analysis
RANSAC:	random sample consensus
SL:	structured light
THP:	two half-planes model
ToF:	time-of-flight

List of Symbols and Notations

$\ \quad \ $	the norm of a vector
$[]$	the place holder for the name of a quantity, a method, or a process
$[]_{th}$	the threshold value
$\overline{\mathbf{p}_i^k}$	the centroid of $\mathbf{P}_i^k$, $\overline{\mathbf{p}_i^k} \in \mathbb{R}^3$
$\mathbf{C}_{N_i}^k$	the covariance matrix of N_i^k , $\mathbf{C}_{N_i}^k \in \mathbb{R}^{3 \times 3}$
$\mathbf{C}_i^k$	the covariance matrix of $\mathbf{P}_i^k$, $\mathbf{C}_i^k \in \mathbb{R}^{3 \times 3}$
N_i^k	the normals of $\mathbf{P}_i^k$
$\mathbf{P}[]$	the set of selected points
$\mathbf{P}_i^k$	the neighborhood of $\mathbf{p}_i$ consists of k neighbors
$c_{[],i}$	the estimated curvature for $\mathbf{p}_i$, $c_{[],i} \in \mathbb{R}^{0+}$
$d_{[],i}$	the point spacing of $\mathbf{p}_i$, $d_{[],i} \in \mathbb{R}^{0+}$
e_i	the fitting error of $\mathbf{p}_i$, $e_i \in \mathbb{R}^{0+}$
n_{ITER}	the number of iterations, $n_{ITER} \in \mathbb{Z}^+$
x_i	the number of preserved neighbors for $\mathbf{p}_i$, $x_i \in \mathbb{Z}^*$
$\mathbf{d}_i$	the edge direction estimated for $\mathbf{p}_i$, $\mathbf{d}_i \in \mathbb{R}^3$
$\mathbf{n}_i$	the normal estimated for $\mathbf{p}_i$, $\mathbf{n}_i \in \mathbb{R}^3$
$\mathbf{n}_i^j$	the normal of $\mathbf{p}_i^j$, $\mathbf{n}_i^j \in \mathbb{R}^3$
$\mathbf{p}_i$	the point in 3D space with index i , which has coordinates (x_i, y_i, z_i) in the Cartesian coordinate system, $\mathbf{p}_i \in \mathbb{R}^3$
$\mathbf{p}_i^j$	the j^{th} nearest neighbor of $\mathbf{p}_i$, $\mathbf{p}_i^j \in \mathbb{R}^3$
$\mathbf{v}'_i$	the i^{th} principal component (i.e. eigenvector) of $\mathbf{C}_{N_i}^k$, $\mathbf{v}'_i \in \mathbb{R}^3$
$\mathbf{v}_i$	the i^{th} principal component (i.e. eigenvector) of $\mathbf{C}_i^k$, $\mathbf{v}_i \in \mathbb{R}^3$

θ_i	the included angle of the two half-planes model of $\mathbf{p}_i$, $0 \leq \theta_i \leq \pi$
λ'_i	the i^{th} eigenvalue of $C_{N_i}^k$, $\lambda'_i \in \mathbb{R}^{0+}$
λ_i	the i^{th} eigenvalue of C_i^k , $\lambda_i \in \mathbb{R}^{0+}$
$\times$	the cross product of two vectors
$\cdot$	the dot product of two vectors
$halfplane(\mathbf{p}_i, \mathbf{p}_i^{j_1}, \mathbf{p}_i^{j_2})$	the half-plane constructed by a query point $\mathbf{p}_i$ and two neighbor points $\mathbf{p}_i^{j_1}$ and $\mathbf{p}_i^{j_2}$
$halfplane(\mathbf{p}_i, \mathbf{p}_i^j, \mathbf{d}_i)$	the half-plane constructed by a query point $\mathbf{p}_i$, a neighbor point $\mathbf{p}_i^j$, and its edge direction $\mathbf{d}_i$
Θ	the included angles of P
$C(a, b)$	the number of combinations of b samples from a items
D	the edge directions of P
E	the fitting errors of P
P	the point cloud $\{\mathbf{p}_i \mid i = 1, 2, 3, \dots, n\}$
$P(a, b)$	the number of permutations of b samples from a items
X	the numbers of preserved neighbors for P
k	the size of a neighborhood
n	the size of a point cloud
$plane(\mathbf{v}_1, \mathbf{v}_2)$	the plane spanned by two vectors $\mathbf{v}_1, \mathbf{v}_2$
$plane(\mathbf{p}_i, \mathbf{v})$	the plane constructed by a point $\mathbf{p}_i$ and a vector $\mathbf{v}$ perpendicular to the plane
α	the distance ratio used by ANMS, $\alpha \in \mathbb{R}^{0+}$
β	the standard deviation multiplier for outlier removal

Chapter 1

1 Introduction

1.1 Motivation

Point cloud, i.e. collection of three-dimensional (3D) points, is a popular medium for the booming 3D sensing technology [1]–[5]. It finds wide applications such as flexible manufacturing [6], quality control [7], and consumer electronics [8].

Edges are curves along which surface directions change abruptly [9]. As shown in Figure 1-1, point clouds represent edges by edge points, i.e. points that are on or sufficiently near (determined by application and sensor characteristics) the edges. Naturally, the rest of the points in the point cloud are referred to as non-edge points. Edge points are crucial for analyzing point clouds. They are used in connection with point cloud processing procedures such as segmentation [10], resampling [11], and mesh generation [12]. Accurate detection of edge points is the foundation for tasks including tool path planning [13], object modeling [11], and bin picking [14].

Conversion-based [14]–[16] detectors and geometry-based [13], [17], [18] detectors are the two mainstream approaches to detect edge points from point clouds. Conversion-based detectors rely on converting point cloud to another representation (such as depth images [14], [15], [19] and

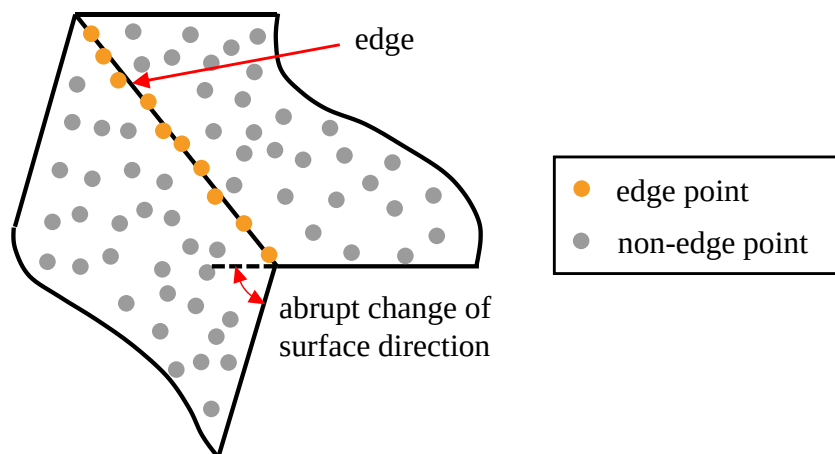


Figure 1-1. Edges are curves along which surface directions change abruptly. Edge points are points in a point cloud that are on or sufficiently near the edges.

triangulated meshes [16]). Detection can be performed in the new representations with well-studied edge detection techniques [14]–[16], [19] and then cast back onto point clouds. However, edge is a type of high-frequency features and is difficult to preserve in conversions. For example, conversions to triangulated meshes tend to smooth out edges, and conversions to depth images rasterize edges in the point cloud.

Geometry-based techniques utilize the neighborhood of each point to derive geometrical properties (e.g. curvature [13], [17] and normal [18]). The derived properties help differentiate edge points from the rest (e.g. curvature of an edge point is usually higher than that of a non-edge point). Using user-defined thresholds, edge points can be predicted from the point cloud. Such techniques have the benefit of direct point cloud usage, which do not incur conversion losses on edge information. However, geometry-based detectors suffers from two limitations as illustrated in Figure 1-2: (1) missed detections (i.e. some edge points are not detected) and (2) redundant detections (i.e. some non-edge points are detected as edge points), especially near edges.

Missed detections stem from using oversized thresholds. When the user-provided threshold is larger than the optimum one, some edge points cannot be detected. This is because the optimum thresholds of the existing geometry-based detectors are dependent on edge angles. For example,

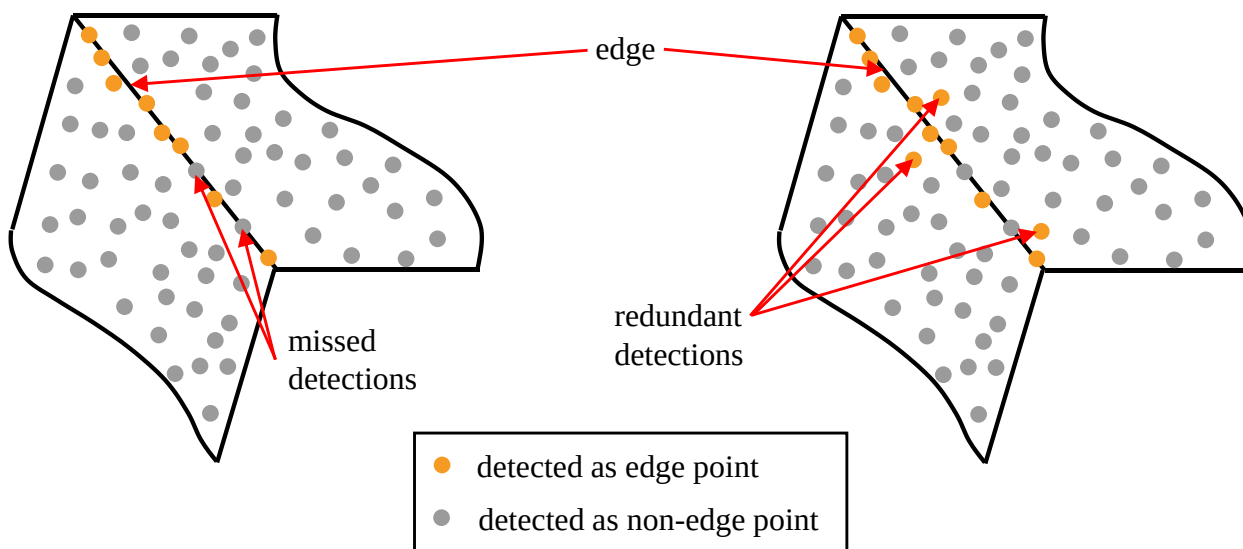


Figure 1-2. Missed detections are undetected edge points. Redundant detections are non-edge points detected as edge points.

as shown in Figure 1-3, a threshold value of 0.4 is suitable for a 90° edge but is too large to detect the 150° edge. Due to the edge angle dependency, a fixed threshold only works for a narrow range of edge angles (e.g. $80^\circ \sim 100^\circ$). While this is suitable for applications involving point clouds with narrow edge angle ranges (e.g. point cloud from a cube [17], or plates perpendicular to each other [13]), edge angles in a general point cloud can vary in a wide range (e.g. $30^\circ \sim 150^\circ$, from an industrial part). This leads to incorrect thresholds for some edge points, resulting in missed detections.

Redundant detections are caused by two reasons. First, when using undersized thresholds, points near edges will be detected. The cause is similar to that of missed detections: the optimum thresholds are dependent on edge angles. Second, geometrical quantities of edge points and their neighbors are similar and cannot be differentiated by thresholds. The problem of redundant detections is overlooked by existing geometry-based detectors possibly because it has little or no effect on their rendering results. However, in industrial applications, such as tool path extraction, it is vital to obtain edge points without the redundant ones, such that precise paths can be generated.

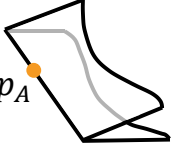
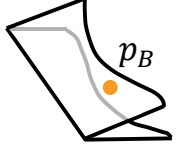
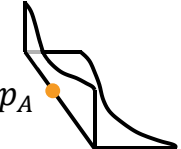
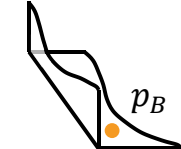
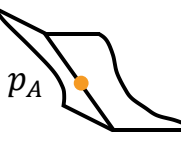
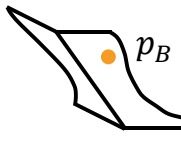
edge angle	edge point p_A and its geometrical quantity	non-edge point p_B and its geometrical quantity	example threshold to differentiate p_A and p_B
30°	 (0.7)	 (0.5)	0.6
90°	 (0.5)	 (0.3)	0.4
150°	 (0.3)	 (0.1)	0.2

Figure 1-3. Thresholds for existing geometrical quantities are dependent on edge angles [13], [17], [18]. Values in brackets are example geometrical quantities estimated by existing geometry-based detectors.

To solve these problems, this thesis presents a new two-stage geometry-based edge detector. First, a novel two half-planes model (THP) is proposed to estimate geometric quantities of point cloud neighborhoods without being affected by edge angles. This makes the threshold required by the detector independent of edge angles, preventing missed detections from oversized thresholds. Second, the traditional non-maximum suppression for 2D grid-like images is adapted for 3D point clouds and added with directional constraints (adapted non-maximum suppression, ANMS). This allows suppression of redundant detections while preserving correctly detected edge points.

1.2 Thesis overview

The remainder of the thesis is organized as follows:

Chapter 2 presents the background of point cloud data, the problem of edge detection in point clouds, and two main approaches of edge detection in point cloud: conversion-based method and geometry-based method.

Chapter 3 proposes a new technique to avoid missed detections when detecting edges from point clouds. The technique uses a two half-planes model (THP) to avoid threshold dependency on edge angles.

Chapter 4 proposes a novel technique to reduce redundant detections when detecting edges from point clouds. This technique adapts the non-maximum suppression method originally developed for 2D image processing. The adapted non-maximum suppression (ANMS) method constructs directional neighborhoods and conduct comparisons on two half-planes fitting errors.

Chapter 5 evaluates the performance of the two proposed techniques on point clouds of different edge angles. The overall performance of the THP+ANMS detector is evaluated with point clouds of multiple industrial parts. The results are compared with existing detectors in terms of detection accuracy, thresholds used, and detection speed.

Chapter 6 concludes the thesis by summarizing contributions and discussing future directions.

Chapter 2

2 Background

2.1 Point cloud and its sources

A point cloud P is a set of points $\mathbf{p}_i \in \mathbb{R}^3$ reside within 3D space, often sampled from surfaces from a scene or an object. A point cloud with n points can be written as $P = \{\mathbf{p}_i \mid i = 1, 2, 3, \dots, n\}$. Usually, each point within a point cloud has a tuple of its X, Y, and Z coordinates expressed in the Cartesian coordinate system, $\mathbf{p}_i = (x_i, y_i, z_i)$. Optionally, each point can have extra attributes such as color and normal direction. For example, a point with red, green, and blue color attributes can be expressed as $\mathbf{p}_i = (x_i, y_i, z_i, r_i, g_i, b_i)$.

Point clouds can be divided into organized point clouds and unorganized point clouds based on their point-wise relationships. In organized (also called structured) point clouds, the points are stored in a 2D grid-like structure with a certain width and length [20]. Points in organized point clouds are implicitly connected to other points in the same row or column. On the other hand, the points in unorganized (also called unstructured) point clouds are stored as individual elements in a set without additional point-wise relationships [21]. Any relationships among points need to be derived from their coordinates and/or extra attributes.

This work considers unorganized point clouds without extra attributes since they belong to the most generalized and basic form of point clouds. Point clouds with extra attributes and organized point clouds can be reduced to this form by stripping away the extra attributes or the grid-like structure. This form is also available from most 3D scanners and allows methods developed for it applicable to other forms of point clouds.

Point clouds are usually created with a 3D scanner that provides a discrete sampling of object surfaces. Common 3D scanners use various principals to generate point clouds, such as physical contact (e.g. a coordinate measurement machine (CMM) [22]), triangulation (e.g. structured light scanner (SL) [2]), and time-of-flight (e.g. time-of-flight camera (ToF) [23]). 3D scanners produce point clouds that suffer from noise (e.g. from the sensing hardware), nonuniform density (i.e. distance from a point to its neighboring points, usually affected by sensing distance and

incident angle), incompleteness (e.g. limited field of view, or obscured by another part of the object), and outliers (e.g. due to interreflection between two surfaces). The common size of a point cloud obtained from a 3D scanner can range from hundreds of points (e.g. from a coordinate measurement machine) to millions of points (e.g. from a laser scanner).

Point clouds can also be artificially generated, given that they are simply a set of 3D coordinates. Usually, the points are not arbitrarily created, but sampled by virtual scanning [24]. In virtual scanning, numerous rays from the scanner are cast onto a virtual object to simulate the scanning process. The intersection points between each ray and the object are calculated via ray tracing algorithms [25]. The artificially generated point clouds contain these intersection points. Optionally, noises can be simulated and added to the coordinates of each point.

Other 3D representations, such as depth image, triangulations, and CAD models, can be converted into point clouds [26]. However, conversions are usually associated with information loss and require additional extrinsic parameters. For instance, when converting a triangulation to point cloud, faces are stripped away and only the vertices are retained. Also, when converting a CAD model to point cloud, the parameterized geometries are discarded while additional parameters such as the desired number of points need to be provided by the user.

2.2 Edge in point clouds

Edges are curves along which surface directions change abruptly [9]. On the surface of an object, there are locations where surface directions (e.g. represented by surface normals) change abruptly (Figure 1-1). Such locations can form points (e.g. the apex of a cone) or curves (e.g. the base of a cone). Edges are curves made up of such locations.

Point clouds represent edges by edge points, i.e. points that are on or sufficiently near the edges (Figure 1-1). Whether a point is sufficiently near the edges or not varies by application. For example, in the application of tool path planning where detected edge points are used to construct tool paths, it is desired that the planned path has little deviation from a target path. The subset of points that offers the minimum path deviation is considered as edge points. The rest of the points in the point cloud are referred to as non-edge points. Note that the notion of edge points differs from boundaries points, which are located on the boundaries of point clouds [27].

2.3 Conversion-based edge detection

Conversion-based detectors [14]–[16] and geometry-based detectors [13], [17], [18] are the two mainstream approaches to detect edge points from point clouds. This section discusses conversion-based detectors, and geometry-based detectors are covered in the next section.

Conversion-based detectors utilize well-studied edge detection techniques for other 3D representations. Detection can be performed in the new representations (e.g. mesh [16] and depth image [14], [15]) and the detection result can be cast back onto point clouds.

Mesheres represent 3D objects by a set of vertices, edges (edges that link two vertices, not essentially the edge we desire to detect), and faces. Triangle mesh is the most commonly used mesh in which each triangle is formed by three vertices [26]. Edge detection in meshes benefits from the connecting information among the vertices, edges, and faces. Normal of each face is also readily available, which can be used to detect edge [28]–[30]. However, algorithms that convert a point cloud to a mesh can be time-consuming (e.g. Delaunay triangulation in 3D has a time complexity of $O(n^2)$ [31]), which prevents this approach from being applied to large point clouds (e.g. with millions of points).

Depth images represent 3D objects by providing depth information in 2D grid-like images. Each pixel within the image stores the distance from the sensing device to a corresponding point on the 3D objects. The major advantage of using depth images is that well-studied image processing techniques for edge detection in 2D images [19], [32] can be readily applied. This is because depth images are essentially a special form of 2D images. However, the conversion from a point cloud to a depth image usually requires the point cloud to be organized (i.e. all points are already stored in a 2D grid-like structure), but most point clouds are unorganized. It is also possible to rasterize an unorganized point cloud into a depth image, but the rasterization process introduces discretization error, requires extrinsic parameters (e.g. choice of viewpoint), and can only partially represent point clouds with overlapping regions viewing from the viewpoint. Although edge detection methods for 2D images can be extended to 3D in voxel representation [33]–[35], voxelization a point cloud also has similar limitations, such as the introduction of discretization error.

Although conversion-based approaches allow us to transform the task of edge detection into another representation, the conversion itself can be time-consuming, introduce errors, or require special conditions. The next section will cover geometry-based methods that circumvent these limitations by directly detecting edges from point cloud neighborhoods.

2.4 Geometry-based edge detection

Geometry-based techniques utilize neighborhood of each point to derive geometrical properties (e.g. curvature and normal). The derived properties help differentiate edge points from the rest (e.g. curvature of an edge point is usually higher than that of a non-edge point). Using user-defined thresholds, edge points can be predicted from the point cloud, as illustrated in Figure 1-3. This section will first introduce the concept of query points and neighborhoods, then summarize techniques for analyzing neighborhoods, and finally review two mainstream geometry-based edge detectors.

2.4.1 Query points and neighborhoods

Unorganized point clouds are usually analyzed within local neighborhoods found by given query points [36], [37] since explicit point-wise relationships are not available. Algorithms have been developed based on neighborhoods to derive locally determined geometrical quantities [38]–[41].

To investigate each point within a point cloud P , a point $\mathbf{p}_i$ is first chosen as a query point. The neighborhood P_i^k of the query point $\mathbf{p}_i$ is a subset of P that contains points near $\mathbf{p}_i$. Such neighborhood can be found by k nearest neighbor (kNN) search [36], [37] or radius neighbor search [42], [43]. kNN search algorithms [36], [37] find k points within P that are nearest to $\mathbf{p}_i$. k specifies neighborhood size (the number of neighbors) and the neighborhood of $\mathbf{p}_i$ is expressed as $P_i^k = \{\mathbf{p}_i^j \mid \mathbf{p}_i^j \text{ is the } j^{th} \text{ nearest neighbor of } \mathbf{p}_i\}$. Radius neighbor search algorithms [42] find points within P whose distances to the query point $\mathbf{p}_i$ are less than a specified radius r : $P_i^k = \{\mathbf{p}_i^j \mid \|\mathbf{p}_i^j - \mathbf{p}_i\| < r\}$. k , in this case, does not affect the search result and is only used to represent the neighborhood size. Efficient methods such as kd-tree [37] and FLANN [36] have been developed to facilitate nearest neighbor search.

Point clouds with extra structures or attributes allow different search approaches. For example, in organized point cloud, the neighbors can be found by collecting points in nearby rows and columns of the query point, which only takes constant time. In point cloud extracted from the vertices of a triangle mesh, neighbors can be found with search methods designed for graphs that utilize the connections provided by edges and faces in the mesh [44]. If available, such characteristics of point clouds can be used to accelerate nearest neighbor search and improve the performance of point cloud processing algorithms.

2.4.2 Techniques for analyzing point cloud neighborhoods

This section covers the techniques commonly used to analyze point cloud neighborhoods, including neighborhood centroid, principal component analysis, and the estimation of point spacing. These techniques serve as the foundation for the existing geometry-based edge detectors.

Neighborhood centroid

The centroid $\overline{\mathbf{p}}_i^k$ of a neighborhood P_i^k is the mean of the points in P_i^k . It can be directly calculated by taking the average of the X, Y, and Z coordinates of the points, given by

$$\overline{\mathbf{p}}_i^k = \frac{1}{k} \cdot \sum_{j=1}^k \mathbf{p}_i^j \quad (2.1)$$

Neighborhood centroid is used in the mean shift edge detector where the distance between query points and their neighborhood centroids are used to identify edge points [13]. In principal component analysis [45]–[47], the neighborhood centroid is used to calculate the covariance matrix of the neighborhood.

Principal component analysis of a point cloud neighborhood

Principal component analysis (PCA) is a method commonly used to identify representative components and to reduce the dimension of a dataset [45]–[47]. By choosing a subset of principal components with the largest eigenvalues, data resides in a high dimensional space can be projected to a lower dimension one. To analyze a point cloud neighborhood P_i^k , PCA can be conducted by performing eigendecomposition on the covariance matrix C_i^k of P_i^k :

$$C_i^k = \frac{1}{k} \sum_{j=i}^k (\mathbf{p}_i^j - \overline{\mathbf{p}_i^k}) \cdot (\mathbf{p}_i^j - \overline{\mathbf{p}_i^k})^T \quad (2.2)$$

The covariance matrix $C_i^k \in \mathbb{R}^{3 \times 3}$ is then decomposed into an orthogonal basis formed by eigenvectors (i.e. principal components) $\{\mathbf{v}_i \in \mathbb{R}^3 \mid i = 0, 1, 2\}$ and eigenvalues along the eigenvectors $\{\lambda_i \mid i = 0, 1, 2\}$:

$$\begin{cases} C_i^k \mathbf{v}_0 = \lambda_0 \mathbf{v}_0 \\ C_i^k \mathbf{v}_1 = \lambda_1 \mathbf{v}_1 \\ C_i^k \mathbf{v}_2 = \lambda_2 \mathbf{v}_2 \end{cases} \quad (2.3)$$

The eigenvectors are usually sorted, such that $0 \leq \lambda_0 \leq \lambda_1 \leq \lambda_2$. $\mathbf{v}_i$ and λ_i are associated if they have the same subscript, and λ_i represent the data variance along $\mathbf{v}_i$. The eigenvalues are nonnegative real numbers because the covariance matrix C_i^k is both symmetric and positive semi-definite.

The eigenvalues and eigenvectors can be used to estimate geometrical quantities of the neighborhood. For example, the plane $plane(\mathbf{v}_1, \mathbf{v}_2)$ spanned by $\{\mathbf{v}_1, \mathbf{v}_2\}$ corresponds to a least-square fitted plane for neighborhood P_i^k [48], and can be used to estimate the normal vector for P_i^k . Also, the ratio among eigenvalues can be used to estimate curvature for P_i^k [17], [41].

Point cloud spacing estimation

The spacing of a point cloud quantifies how close points are to each other around a query point. It is used by point cloud processing methods since it allows quantities to be made dimensionless [13].

The commonly used kNN estimator [49] calculates local spacing with the average distance from a query point to its neighbors, given by

$$d_{kNN_spacing,i} = \frac{1}{k} \sum_{j=1}^k \|\mathbf{p}_i^j - \mathbf{p}_i\| \quad (2.4)$$

kNN estimators tend to assign larger spacing to points near point cloud boundary since their neighbor points all reside within the same direction [27]. Although a small k mitigates the boundary bias [13], the estimated spacing is heavily influenced by points that are occasionally too close to each other.

In the field of geographic information system (GIS), point density is estimated by counting the number of points in a user-defined grid [50], quantifying how dense the points are sampled. Since the point cloud used in GIS is usually a survey of some landscapes, a grid can be created on top of the horizontal plane. The density is calculated in terms of the number of points that fall into the area of a grid. Point spacing can be converted from point density using the relationship specified in [50]:

$$d_{spacing,i} = \sqrt{\frac{1}{d_{density,i}}} \quad (2.5)$$

2.4.3 Existing geometry-based detectors

Curvature-based edge detection

The curvature of a 3D surface quantifies how much the surface locally deviates from being a flat plane [51]. However, traditional curvature estimation methods for 3D surfaces cannot be applied to point clouds. Methods have been developed for estimating curvature within point clouds [13], [41], usually quantifying the deviation from a neighborhood to a flat plane.

Curvature estimated for a query point can be used to indicate whether it is an edge point. Query points positioned on planar neighborhoods will have lower curvatures than those located near an edge (Figure 2-1). This means points with higher curvatures are more likely to be edge points.

When a user-defined threshold c_{th} is applied, the detected edge points are given as

$$P_{detected_edge} = \{\mathbf{p}_i \mid c_i > c_{th} \text{ and } \mathbf{p}_i \in P\} \quad (2.6)$$

where c_i is the curvature estimated for $\mathbf{p}_i$.

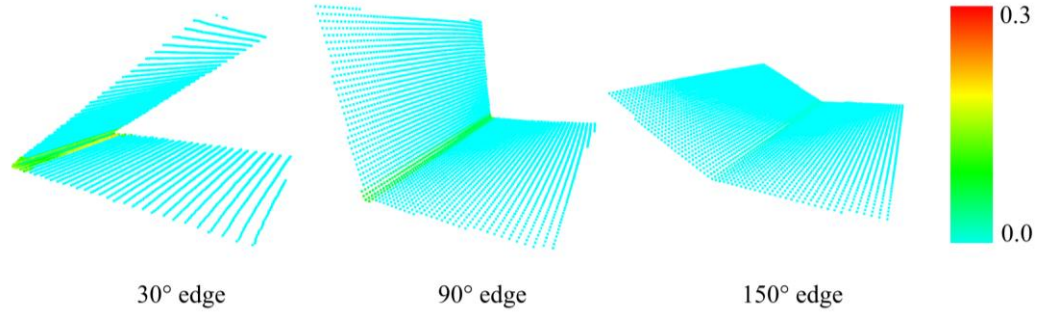


Figure 2-1. Curvature estimated with surface variation of point clouds with various edge angles. The color bar shows the magnitude of curvatures.

Various approaches have been developed to estimate curvature c_i for a query point p_i based on its neighborhood. The curvature of a query point can be estimated with the ratio among eigenvalues obtain by the PCA result of its neighborhood. For example, assuming a flat plane $plane(\mathbf{v}_1, \mathbf{v}_2)$ is found by spanning the principal components $\{\mathbf{v}_1, \mathbf{v}_2\}$ associated with the largest two eigenvalues $\{\lambda_1, \lambda_2\}$. The smallest eigenvalue λ_0 quantifies how much the neighborhood P_i^k deviates from the plane, since it is the variance of orthogonal distance from each neighbor $\mathbf{p}_i^j \in P_i^k$ to $plane(\mathbf{v}_1, \mathbf{v}_2)$. λ_0 , being the variance of distance has a unit of squared length (L^2 , e.g. mm^2), and depends on the spacing of the neighborhood. To make λ_0 dimensionless, [41] suggests using the ratio of λ_0 to the sum of eigenvalues $\{\lambda_i \mid i = 0, 1, 2\}$ as curvature $c_{surface_variation,i}$ for $\mathbf{p}_i$:

$$c_{surface_variation,i} = \frac{\lambda_0}{\lambda_0 + \lambda_1 + \lambda_2} \quad (2.7)$$

This curvature estimated from the eigenvalue ratio is originally named surface variation in [41] and later referred to as an estimation for curvatures [17], [38]. Improvements on $c_{surface_variation,i}$ estimation includes, for example, using a subset of neighbors for robust estimation [38] and using weights to limit influence from outliers [1].

Curvature can also be estimated by calculating the mean shift [13], which measures the Euclidean distance $d_{mean_shift,i}$ between a query point $\mathbf{p}_i$ and the centroid of its neighborhood $\bar{\mathbf{p}}_i$. To make the distance comparable across point clouds with nonuniform spacing, [13] divide

$d_{mean_shift,i}$ with spacing estimated by the distance between $\mathbf{p}_i$ and its first nearest neighbor $\mathbf{p}_i^1$. The curvature estimated by mean shift $c_{mean_shift,i}$ is expressed as:

$$c_{mean_shift,i} = \frac{d_{mean_shift,i}}{d_{spacing,i}} = \frac{\|\mathbf{p}_i - \overline{\mathbf{p}}_i^k\|}{\min_{j=1,2,3,\dots,k} \|\mathbf{p}_i - \mathbf{p}_i^j\|} = \frac{\|\mathbf{p}_i - \overline{\mathbf{p}}_i^k\|}{\|\mathbf{p}_i - \mathbf{p}_i^1\|} \quad (2.8)$$

Normal-based edge detection

Normal-based detectors rely on the abrupt change of surface directions around edge points, as surface directions can be represented by normals. Normals can be estimated from the coordinates of three points [18]. For example, from a given query point $\mathbf{p}_i$ and two of its neighbors $\mathbf{p}_i^{j_1}, \mathbf{p}_i^{j_2} \in P_i^k$, the normal $\mathbf{n}_i$ can be found by

$$\mathbf{n}_i = \frac{(\mathbf{p}_i - \mathbf{p}_i^{j_1}) \times (\mathbf{p}_i - \mathbf{p}_i^{j_2})}{\|(\mathbf{p}_i - \mathbf{p}_i^{j_1}) \times (\mathbf{p}_i - \mathbf{p}_i^{j_2})\|} \quad (2.9)$$

PCA-based approaches estimate normal with the eigenvector $\mathbf{v}_0$ associated with the smallest eigenvalue λ_0 [46]. The neighborhood can be fitted with a plane $plane(\mathbf{v}_1, \mathbf{v}_2)$ spanned by the other two eigenvectors $\{\mathbf{v}_1, \mathbf{v}_2\}$, and $\mathbf{v}_0$ is perpendicular to $\mathbf{v}_1$ and $\mathbf{v}_2$. Other normal estimation methods are based on direct plane-fitting, such as random sample consensus (RANSAC) [52]–[54] and Hough transform (HT) [54]–[56].

To detect edge points from estimated normals, [18] proposes to investigate the clustering result of normals within a neighborhood as shown in Figure 2-2.

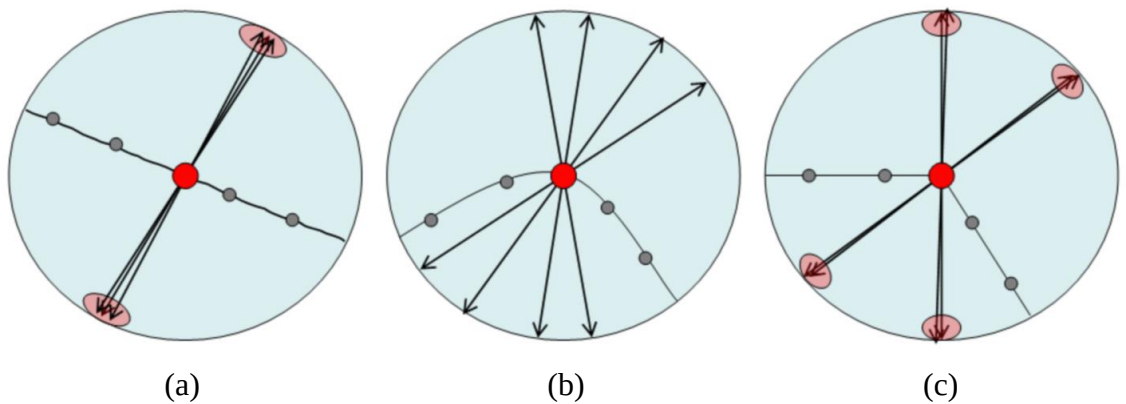


Figure 2-2. Possible normal clustering results. Reproduced from [18] (CC BY-NC-ND 3.0).

The clustering results can be divided into three categories, as indicated in Figure 2-2: (a) If a neighborhood is on a flat surface, the normals within the neighborhood should belong to a single cluster. Since signs of normals are ambiguous, normals with opposite directions are considered as close to each other. (b) If a neighborhood spreads across an edge, the normals in the neighborhood should be separated into two clusters, each cluster for one side of the edge. And (c) if a neighborhood is on a smooth non-planar surface, the normals within the neighborhood should not be able to form any significant clusters. The normals calculated by (2.9) are clustered with the hierarchical agglomerative clustering (HAC) algorithm [57], [58], with a user-defined threshold distance as the terminating condition for the clustering process.

Chapter 3

3 Two half-planes model to avoid missed detections

In this chapter, we first analyze the problem of missed detections from the perspective of oversized thresholds. Existing methods to solve missed detections are dependent on edge angles, which are unknown during detection and can vary largely in a point cloud. We then present a novel geometry-based method that employs two half-planes model (THP) to avoid missed detections without dependency on edge angles.

3.1 The problem of missed detections

For a geometry-based edge detector, missed detections are the problem of marking edge points as non-edge points, i.e. the inability to detect edge points (Figure 1-2). Geometry-based detectors rely on user-defined thresholds, and the choice of threshold values largely affects the detection result. The problem of missed detections roots from oversized thresholds applied to the estimated geometrical quantities.

Geometry-based detectors decide whether a query point is an edge point or not based on its neighborhood. Geometrical quantities (e.g. curvature and normal) are estimated from the neighborhood and can be used to indicate how likely the query point is an edge point. For example, as shown in Figure 2-1, a query point far away from the edge whose neighborhood represents a flat plane will have a low curvature, indicating the query point is unlikely to be an edge point. On the other hand, if the query point is close to an edge and its neighborhood consists of two sides of the edge, a high curvature will be calculated, indicating the query point is likely an edge point. A properly chosen curvature threshold can be used to identify the edge points. Missed detections occur when using oversized thresholds, since edge point with geometrical quantities lower than the thresholds cannot be detected.

3.2 Existing solutions for missed detections

To avoid oversized thresholds, existing geometry-based methods use manually tuned thresholds that are suitable for the edge angles. Such methods are suitable for point clouds whose edge angles are within a narrow range (e.g. $80^\circ \sim 100^\circ$, from the point cloud of a cube). This is because,

in such point clouds, the range of geometrical quantities of edge points and the range of non-edge points can be separated with a properly chosen threshold (Figure 1-3).

However, not all point clouds have edge angles that fall within a narrow range. Point clouds, in general, can have edge angles that span a wide range (e.g. $30^\circ \sim 150^\circ$, from the point cloud of an industrial part). With such point clouds, the chosen threshold can only be used to detect edge points within the narrow range. For edges with angles larger than the narrow range (e.g. larger than 100°), the chosen threshold is oversized and larger than the geometrical quantities of some edge points, so missed detections will occur. On the other hand, for edges with angles smaller than the narrow range (e.g. smaller than 80°), the chosen threshold is undersized. Although undersized thresholds do not incur missed detections, non-edge points near edges will be detected, and redundant detections will occur. The problem of redundant detections is discussed in Chapter 4.

Traditionally, thresholds are only applicable to a narrow range of edge angles. Existing geometrical quantities are calculated based on the neighborhoods of a query point, and the distribution of neighbors within the neighborhoods is affected by edge angles. For instance, the distribution of a neighborhood whose edge angle is near 180° is similar to a flat plane, resulting in relatively low curvature. In contrast, the distribution of a neighborhood whose edge angle is 90° is very different from a flat plane, resulting in a relatively high curvature.

3.3 Two half-planes model

We propose to fit a two half-planes model (THP) for each point cloud neighborhood, as shown in Figure 3-1. THP can be used to locally represent an edge: each half-plane represents one side of the edge, the intersection line of the half-planes represents the edge position and direction, and the included angle between the half-planes represents the edge angle.

Different from existing geometry-based detectors, since the edge angle can be explicitly modeled by the included angle between two half-planes, the fitting error is not affected by edge angles. The fitting error reflects how well a neighborhood fits an edge locally represented by THP. For example, if the query point is near an edge, the fitted two half-planes will have a low fitting error (Figure 3-1 (a)). On the other hand, if the query point does not locate near an edge, the fitted two half-planes will have a high fitting error (Figure 3-1 (b)).

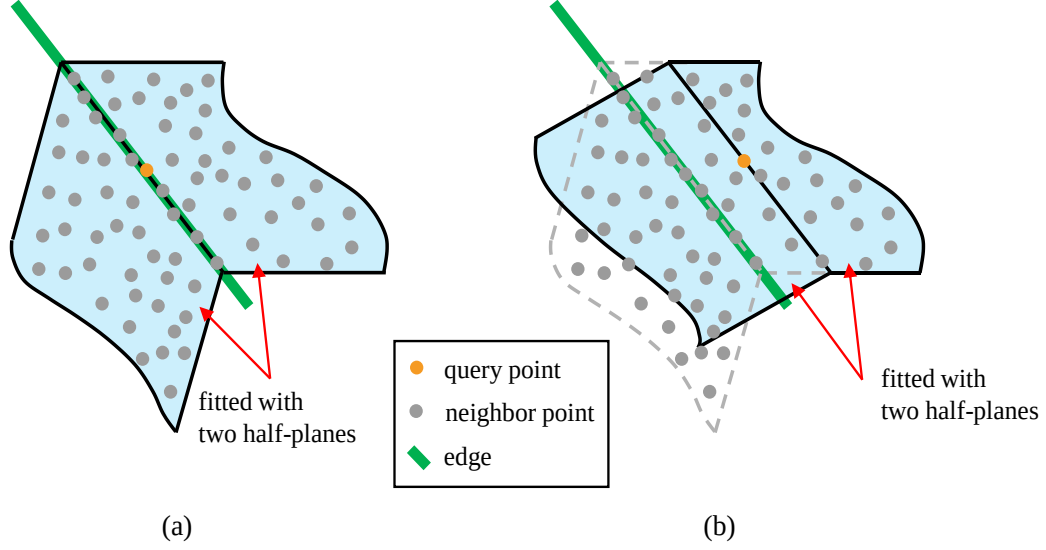


Figure 3-1. Two half-planes model (THP). (a) the fitted two half-planes are close to the neighbor points, results in a low THP fitting error. (b) the fitted two half-planes deviate from the neighbor points, results in a high THP fitting error.

The process of two half-planes fitting is shown in Figure 3-2. We take a point cloud as input containing edge points we desire to detect. The neighborhood of a query point is then found. The fitting result of each query point contains a fitting error and parameters of the two half-planes. The fitting results are further analyzed to provide potential edge points. Usually, every point within the point cloud is used as the query point for the following steps.

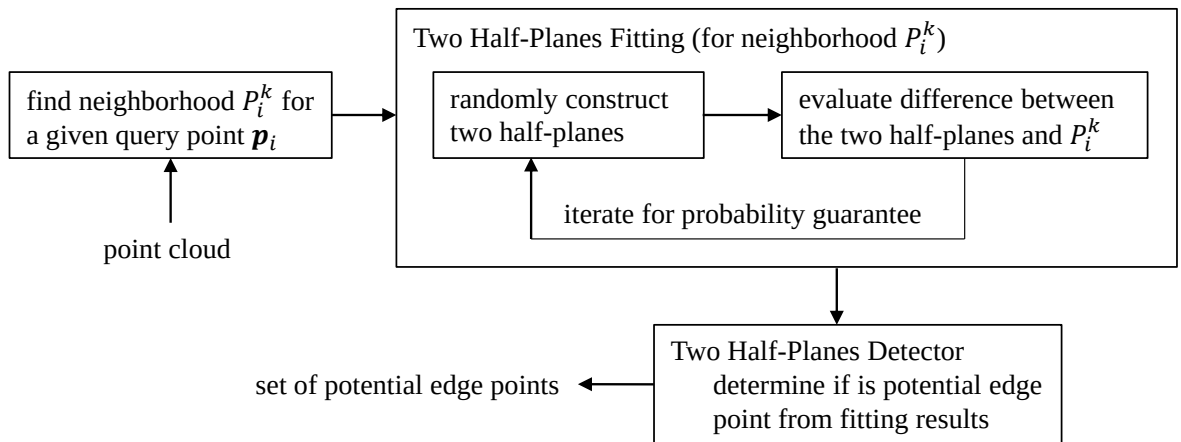
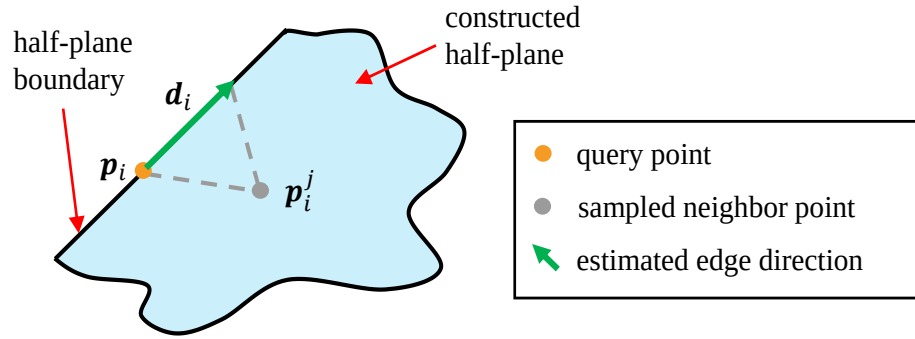


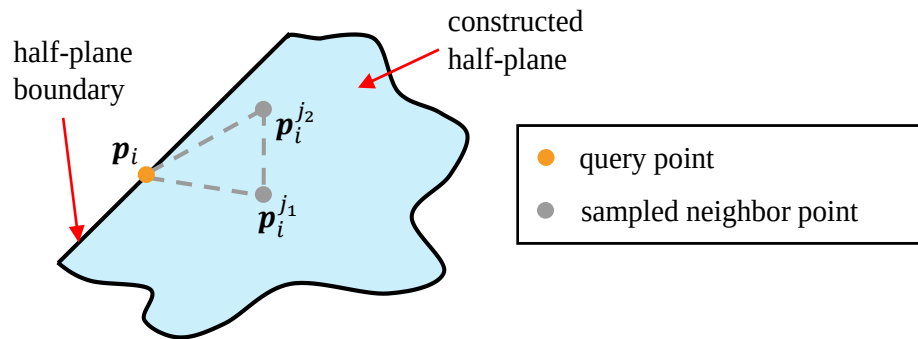
Figure 3-2. The process of two half-planes model (THP) fitting for potential edge point detection. Each point within the point cloud is processed by fitting two half-planes to its neighborhood. The fitting results are then used to select potential edge points for further processing.

Specifically, we first obtain the neighborhood of a query point with the nearest neighbors search. To facilitate this process, k nearest neighbors search [36], [37] is performed. The value of neighborhood size k is provided by the user and depends on application types.

We then fit the neighborhood with two half-planes. The fitting error and parameters of the half-planes are obtained. We construct two half-planes from the neighborhood and evaluate how well the constructed half-planes represent the neighborhood by calculating a fitting error. With traditional plane fitting techniques [52]–[56], when a plane is being fitted to one side of the edge, the points from the other side of the edge function as outliers and affect the fitting result. We avoid this problem by constructing the two half-planes with Monte Carlo methods [59], [60]. We first randomly sample two points from the neighborhood. Each half-plane is constructed with the sampled points, the query point, and optionally an estimated edge direction (Figure 3-3).



(a)



(b)

Figure 3-3. Half-plane construction. (a) constructed with the query point, one sampled neighbor point, and the estimated edge direction, with half-plane boundary defined by the edge direction. (b) constructed with the query point and two sampled neighbor points, with half-plane boundary defined by the intersection line with another half-plane.

This sample and construct approach will be iterated multiple times to achieve a probability guarantee to represent the neighborhood. A half-plane constructed by the query point $\mathbf{p}_i$, a sampled neighbor point $\mathbf{p}_i^j$, and its edge direction $\mathbf{d}_i$ can be written as $halfplane(\mathbf{p}_i, \mathbf{p}_i^j, \mathbf{d}_i)$, with half-plane boundary determined by the edge direction (Figure 3-3 (a)). A half-plane constructed by the query point $\mathbf{p}_i$ and two sampled neighbor points can be written as $halfplane(\mathbf{p}_i, \mathbf{p}_i^{j_1}, \mathbf{p}_i^{j_2})$, with half-plane boundary determined by the intersection line with another half-plane (Figure 3-3 (b)). The use of edge direction (Figure 3-3 (a)) is to reduce the number of sampled points required for half-plane construction (Figure 3-3 (b)), from two sampled points to one sampled point for each half-plane. A lower number of sampled points requires fewer iterations to achieve the same probability guarantee. The edge direction is estimated by (4.4) and will be further discussed in Chapter 4.

We evaluate the fitting error between the constructed two half-planes and the neighborhood with a dimensionless root mean square of the half-plane distances. We calculate the half-plane distance for each neighbor point. As shown in Figure 3-4, half-plane distance measures the nearest distance from a neighbor point to any point on a half-plane. It is a combination of point-to-plane orthogonal distance and point-to-line orthogonal distance. If the point can be orthogonally projected onto the half-plane, then the point-to-plane distance is used as half-plane distance. Otherwise, the point-to-line distance between the plane and the edge is used.

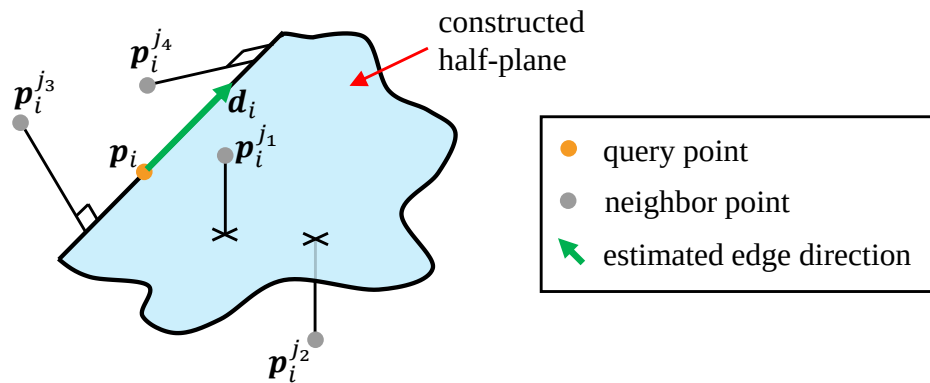


Figure 3-4. Half-plane distance. $\mathbf{p}_i^{j_1}$ and $\mathbf{p}_i^{j_2}$ can be projected onto the half-plane, so point-to-plane distances are used as the half-plane distance. $\mathbf{p}_i^{j_3}$ and $\mathbf{p}_i^{j_4}$ cannot be projected onto the half-plane, so point-to-line distance are used.

Since there are two half-planes involved, we calculate the distance with both half-planes and store the smaller distance as $d_{halfplane,i}^j$. We calculate the root mean square value for $\{d_{halfplane,i}^j \mid j = 1, 2, 3, \dots, n\}$ as:

$$d_{RMS,i} = \sqrt{\frac{1}{k} \sum_{j=1}^k (d_{halfplane,i}^j)^2} \quad (3.1)$$

The root mean squared distance has a length unit and it is desirable to make it dimensionless, such that the fitting error calculated for point clouds with various spacings or are comparable to each other. The mainstream approaches for the estimation of spacing and density are the kNN estimator [49] and the grid-based estimator used in the field of GIS [50]. However, the kNN estimator is affected by neighbor points that are too close or too far from the query point and tend to assign larger value for points near the boundary of point clouds. Also, the GIS grid-based estimator assumes the point cloud is captured from a landscape and can be projected onto a 2D plane, but point clouds of industrial objects rarely lie on a plane. To avoid these drawbacks, we estimate the local spacing of a query point by first calculating the distance $d_{nearest,j}^k$ from each neighbor $\mathbf{p}_j^k$ point to their first nearest neighbor. The nearest distances are then collected within the neighborhood as $\{d_{nearest,j}^k \mid \mathbf{p}_j^k \in P^k\}$, and its median is used to estimate the spacing $d_{spacing,i}$ for the query point (3.2). The spacing is used to scale the root mean squared distance, and a dimensionless fitting error e_i can be obtained by

$$d_{spacing,i} = \text{median}(\{d_{nearest,j}^k \mid \mathbf{p}_j^k \in P^k\}) \quad (3.2)$$

$$e_i = \frac{d_{RMS,i}}{d_{spacing,i}} \quad (3.3)$$

The process of constructing THP and evaluating its fitting error is then repeated multiple times. Although an exhaustive search can be performed by constructing a model for each possible two-point combination from the neighborhood, the number of iteration required would be $n_{ITER} = C(k, 2) = k(k-1)/2$, which would quickly become too expensive to calculate with a large k . For example, a regular k (e.g. $k = 50$) would require 1225 number of iterations. Instead of calculating every two-point combinations, we resort to the Monte Carlo method [59], [60], which

allows us to reduce the number of iterations while providing a probability guarantee. The number of iterations n_{ITER} used by the Monte Carlo method can be calculated based on the desired probability p of successfully finding a model and the probability of choosing the correct points to construct the two half-planes q . The probability of finding a two half-planes model when conducting the process n_{ITER} times can be expressed as

$$1 - p = (1 - q)^{n_{ITER}} \quad (3.4)$$

Thus, the number of iterations can be then calculated by

$$n_{ITER} = \frac{\log(1 - p)}{\log(1 - q)} \quad (3.5)$$

The probability of finding the correct points for constructing the model is unknown before the calculation, but can be calculated as $q = P(2, 2) \cdot q_a q_b = 2 \cdot q_a q_b$, where q_a and q_b are the probability of choosing a neighbor point that belongs to one of the half-planes. In this work, we use $p = 0.99$ and $q_a = q_b = 0.25$, since they work well with point clouds of industrial parts and provide a balance of performance and efficiency. After rounding up, this gives us $n_{ITER} = 35$, much smaller than the 1225 iterations with every possible two-point combinations. Note that if the two half-planes are constructed without an estimated edge direction, four sampled neighbor points will be required (Figure 3-3 (b), each half-plane requires two neighbors). The calculation of q will be $q = \frac{P(4,4)}{P(2,2)^2} \cdot q_a^2 q_b^2 = 6 \cdot q_a^2 q_b^2$. If we use the same values for p , q_a , and q_b , the iteration times will be $n_{ITER} = 195$, requiring more computational resources.

The fitting result for each query point consists of the fitting error and the parameters of the two half-planes. The edge angle can be estimated by the included angle of the half-planes. Based on the fitting result, we filter points in the point cloud to provide a set of potential edge points.

We first filter points whose neighborhood only contains points from a flat surface. Such points have an estimated edge angle close to 0° or 180° . Next, points with fitting errors lower than a user-defined threshold e_{th} are preserved. This is because the fitting error provides an insight on how likely the query point is an edge point: the neighborhood of an edge point can be fitted to the two half-planes with low fitting error, while the neighborhood of a non-edge point incurs high fitting error. The threshold e_{th} need to be manually selected to minimize missed detections,

which can be quantified by the recall value (5.1). When multiple thresholds achieve the same level of missed detection, the value associated with the least false positives can be chosen. The choice of e_{th} is affected by the characteristics of the input point cloud, such as the sampling density and noise level. We experimentally showed in Chapter 5 that the same threshold can be used for point clouds captured from the same scanner, regardless of edge angles.

Unlike existing geometry-based approaches, THP incorporates the angle of an edge, and thus the fitting error is not affected by the magnitude of edge angles. The potential edge points can contain redundant detections and will be removed with the adapted non-maximum suppression technique presented in the next chapter.

The two half-planes fitting method is summarized in Algorithm 3-1 and the fitting result is used to select potential edge points as presented in Algorithm 3-2.

Algorithm 3-1: Two-Half-Planes-Fitting

Input: a point cloud with n points $P = \{\mathbf{p}_i \mid i = 1, 2, 3, \dots, n\}$;
edge directions $D = \{\mathbf{d}_i \mid i = 1, 2, 3, \dots, n\}$, estimated by (4.1) ~ (4.4);
spacing $\{d_{spacing,i} \mid i = 1, 2, 3, \dots, n\}$, estimated by (3.2);
neighborhood size k ;
number of iteration $n_{ITER} \leftarrow (3.5)$.

Output: fitting errors $E = \{e_i \mid i = 1, 2, 3, \dots, n\}$;
included angles of the two half-planes $\Theta = \{\theta_i \mid i = 1, 2, 3, \dots, n\}$.

```

1: Initialize  $E$  with  $e_i \leftarrow +\infty$ 
2: Initialize  $\Theta$  with  $\theta_i \leftarrow 0$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:   find neighborhood  $P_i^k$  from query point  $\mathbf{p}_i$  with kNN search [36], [37]
5:   repeat
6:     randomly sample  $\mathbf{p}_i^x, \mathbf{p}_i^y \in P_i^k$  ( $x, y = 1, 2, 3, \dots, k, x \neq y$ )
7:     if  $(\mathbf{p}_i - \mathbf{p}_i^x)$  or  $(\mathbf{p}_i - \mathbf{p}_i^y)$  is colinear with  $\mathbf{d}_i$ 
8:       continue // otherwise cannot construct half-planes
9:     end if
10:     $hp_1 \leftarrow halfplane(\mathbf{p}_i, \mathbf{p}_i^x, \mathbf{d}_i)$ 
11:     $hp_2 \leftarrow halfplane(\mathbf{p}_i, \mathbf{p}_i^y, \mathbf{d}_i)$ 
12:    for  $j \leftarrow 1$  to  $k$  do
13:       $d_{halfplane,i}^j \leftarrow \min(\text{half-plane distance between } hp_1 \text{ and } \mathbf{p}_i^j,$ 
                                half-plane distance between  $hp_2$  and  $\mathbf{p}_i^j)$ 

```

```

14:   end for
15:   fitting error  $e \leftarrow (3.3)$ 
16:   if  $e < e_i$ 
17:      $e_i \leftarrow e$ 
18:      $\theta_i \leftarrow$  the included angle between  $hp_1$  and  $hp_2$ 
19:   end if
20: until reached  $n_{ITER}$  times
21: end for
22: return  $(E, \Theta)$ 

```

Algorithm 3-2: Two-Half-Planes-Detector

Input: a point cloud with n points $P = \{\mathbf{p}_i \mid i = 1, 2, 3, \dots, n\}$;
fitting errors $E = \{e_i \mid i = 1, 2, 3, \dots, n\}$;
included angles of the two half-planes $\Theta = \{\theta_i \mid i = 1, 2, 3, \dots, n\}$;
fitting error threshold $e_{th} = 0.5$;
angle threshold $\theta_{th} = \pi/15$, to exclude flat planes.

Output: potential edge points selected by two half-planes detector P_{THPD} .

```

1: Initialize  $P_{THPD} \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   if  $\theta_{th} < \theta_i$  and  $\theta_i < (\pi - \theta_{th})$  // the two half-planes do not resemble a flat plane
4:     if  $e_i < e_{th}$ 
5:        $P_{THPD} \leftarrow (P_{THPD} \cup \{\mathbf{p}_i\})$ 
6:     end if
7:   end if
8: end for
9: return  $P_{THPD}$ 

```

Chapter 4

4 Adapted non-maximum suppression to avoid redundant detections

4.1 The problem of redundant detections

For a geometry-based edge point detector, redundant detections are the problem of marking non-edge points near edges as edge points (Figure 1-2). It can be regarded as false positive detections near edges.

The problem can be caused by two reasons. When using undersized thresholds that are smaller than the optimum threshold, points near edges will be detected. As we have analyzed in Chapter 3, this is due to the dependency of the optimum threshold on edge angles and can be avoided with the proposed two half-planes fitting method. The second cause is that geometrical quantities of edge points and non-edge points near edges are similar and cannot be differentiated by thresholds.

4.2 Existing solutions for redundant detections

The problem of redundant detections is often overlooked by geometry-based detectors because existing detectors are mostly used to generate edge points for visualization [13], [17], [18], and redundant detections have little or no effect on the rendering results. Point cloud thinning techniques have been developed to produce new points located at the center of the input point cloud [61]–[63], instead of detecting edge points (Figure 4-1). Such methods are effective in tasks such as abstracting and reconstructing shapes. However, in the problem of redundant detections, edge points usually do not locate at the center. The deviation towards the center is undesirable for industrial tasks. For example, in tool path generation the deviation can cause the tool head to collide with a workpiece.

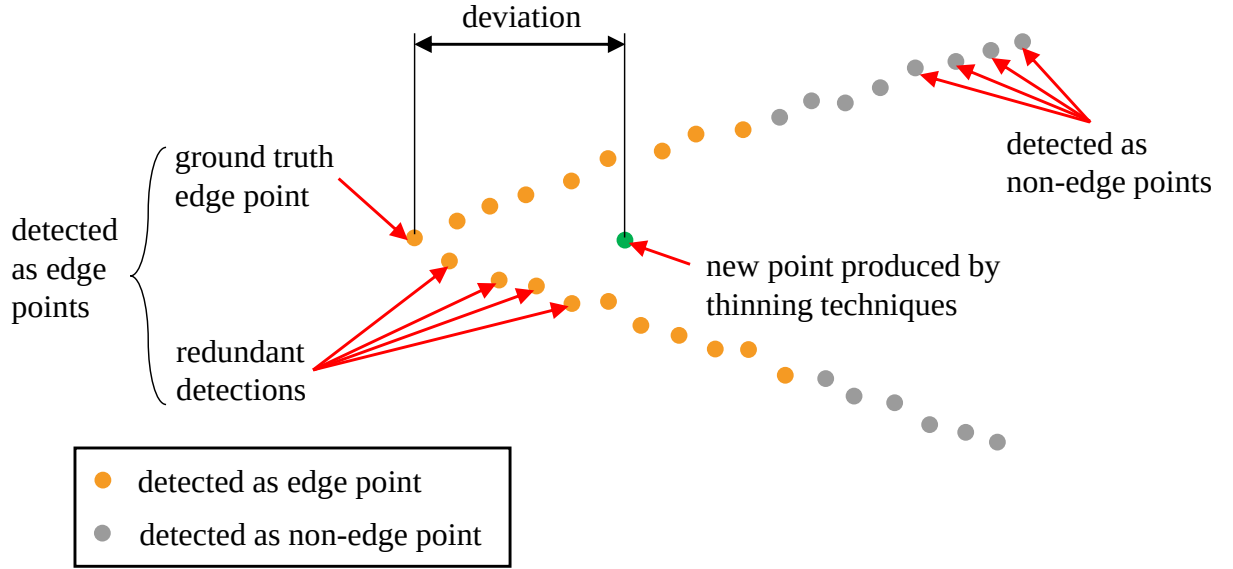


Figure 4-1. The deviation associated with a new point produced by thinning techniques.

In the field of image processing, the problem of redundant detections is solved with non-maximum suppression (NMS) [19], [32]. Non-maximum suppression is originally developed for edge detection in 2D images [19] and has since been applied for 2D image processing tasks such as object detection [64], [65]. Non-maximum suppression algorithms are based on pixel intensities, which allow the calculation of the magnitude and direction of pixel intensity gradients [19], [32]. Different from the point cloud thinning techniques, pixels are locally compared and only those with the maximum intensity gradient magnitude are preserved. However, non-maximum suppression cannot be directly applied for point clouds, because it relies on the magnitude and direction of intensity gradients that are not available in general point clouds.

4.3 Adapted non-maximum suppression

To adapt non-maximum suppression for point clouds, we first review the traditional non-maximum suppression algorithm. Non-maximum suppression reduces redundant detections in 2D grid-like images consist of organized pixels. Pixels that do not have the maximum gradient intensity (i.e. the magnitude of intensity gradient) when comparing to its connected pixels are suppressed. Only the connected pixels that fall in the intensity gradient direction are considered. The steps used by non-maximum suppression are:

Step 1, a directional neighborhood is constructed around a given query pixel

(1a), the horizontal gradient and vertical gradient are calculated.

(1b), the gradient direction is calculated by taking the arctangent of the horizontal intensity gradient and the vertical intensity gradient.

(1c), the gradient direction is rounded (e.g. to multiples of 45°) and masked onto a kernel (e.g. a 3×3 kernel). The masked kernel is then used as the directional neighborhood.

Step 2, the query pixel is suppressed if it does not have the largest gradient intensity within the directional neighborhood.

(2a), for the query pixel and the pixels within the directional neighborhood, their gradient intensities are calculated from the horizontal gradient and vertical gradient.

(2b), the gradient intensities are compared. If the query pixel has the largest gradient intensity, it is preserved. Otherwise, the query pixel is suppressed.

The adapted non-maximum suppression (ANMS) is designed for point clouds with two modifications and one additional operation. First, we replace intensity gradient direction with an estimation of edge direction. The purpose of intensity gradient in traditional non-maximum suppression is to constrain the scope for local gradient intensity comparison. To achieve a similar function in point clouds, we estimate edge direction for each point. As shown in Figure 4-2, the

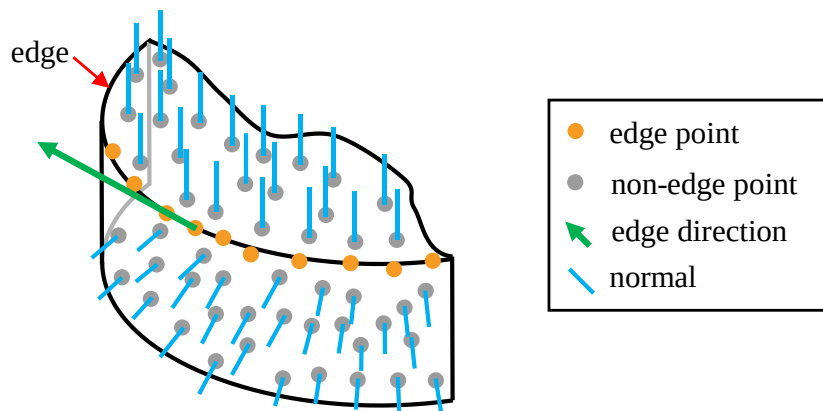


Figure 4-2. The edge direction at a given location is the tangent of an edge and is perpendicular to normals in the neighborhood.

edge direction at a given location is the tangent of an edge, and the normal of surfaces around an edge is perpendicular to the edge direction. Since edge points have not been detected before this process, edge directions are estimated for every point in the point cloud.

Second, instead of comparing gradient intensity that is not usually available in point clouds, we compare geometrical quantities. A comparison of gradient intensity is performed in traditional non-maximum suppression such that only pixels with the highest gradient intensity are preserved. This is because, in 2D images, pixels near edges are associated with high gradient intensity. When multiple pixels have similar intensities, the pixel with the highest gradient intensity is most likely to be a pixel of an edge. To find a replacement for gradient intensities such that similar comparisons can be made in point clouds, we look at available geometrical quantities that allow us to assess how likely a point is an edge point. We observe that when compared with neighboring non-edge points, edge points are associated with low THP fitting error and small estimated angle.

We additionally suppress false positives sparsely distributed far away from the edges. Non-maximum suppression reduces redundant detections by suppressing false positives near edges. Because false positives can also be outliers that are far away from the edges, in ANMS, we suppress these false positives with the outlier removal algorithm [66], [67], which removes potential edge points with a low number of edge point neighbors. We utilize edge direction in this process such that edge points at the end of edges can be preserved. The steps used by ANMS are:

Step 1, a directional neighborhood is constructed around a given query point

(1a), normals of a given neighborhood P_i^k are collected in a normal set N .

Because for each normal n_i , the direction is ambiguous and is biased to one of the two possible directions, both the normal n_i and its flipped counterpart $-n_i$ are collected. The mean of N_i^k , i.e. $\overline{n_i^k}$, is a zero vector since N_i^k contains both signs for each normal. We have

$$N_i^k = \{n_i^j \mid n_i^j \text{ or } -n_i^j \text{ is the normal of } p_i^j \in P_i^k, j = 1, 2, 3, \dots, k\} \quad (4.1)$$

(1b), principal component analysis is performed on the normal set. A covariance matrix $C_{N_i}^k \in \mathbb{R}^3$ is calculated and decomposed into three principal components $\{\mathbf{v}'_0, \mathbf{v}'_1, \mathbf{v}'_2\}$ and the three associated eigenvalues $\{\lambda'_0, \lambda'_1, \lambda'_2\}$ ($0 \leq \lambda'_0 \leq \lambda'_1 \leq \lambda'_2$). The principal component associated with the smallest eigenvalue is used as the estimation for edge direction $\mathbf{d}_i$:

$$\begin{aligned} C_{N_i}^k &= \frac{1}{k} \sum_{j=1}^k (\mathbf{n}_i^j - \overline{\mathbf{n}}^k) \cdot (\mathbf{n}_i^j - \overline{\mathbf{n}}^k)^T \\ &= \frac{1}{k} \sum_{j=1}^k (\mathbf{n}_i^j) \cdot (\mathbf{n}_i^j)^T \end{aligned} \quad (4.2)$$

$$\begin{cases} C_{N_i}^k \mathbf{v}'_0 = \lambda'_0 \mathbf{v}'_0 \\ C_{N_i}^k \mathbf{v}'_1 = \lambda'_1 \mathbf{v}'_1 \\ C_{N_i}^k \mathbf{v}'_2 = \lambda'_2 \mathbf{v}'_2 \end{cases} \quad (4.3)$$

$$\mathbf{d}_i = \mathbf{v}'_0 \quad (4.4)$$

(1c), a directional neighborhood is constructed such that the neighbors are perpendicular to the edge direction. Since neighboring points generally do not fall exactly on the plane perpendicular to the edge direction, neighboring points that are sufficiently near the plane are also counted. The directional neighborhood is given as:

$$P_{directional,i}^k = \{\mathbf{p}_i^j \mid \mathbf{p}_i^j \in P_i^k \text{ and } \|(\mathbf{p}_i - \mathbf{p}_i^j) \cdot \mathbf{d}_i\| < \alpha \cdot d_{spacing,i}\} \quad (4.5)$$

where $d_{spacing,i}$ is the point spacing of $\mathbf{p}_i$ estimated by (3.2), $\alpha \cdot d_{spacing,i}$ is the distance under which a neighboring point is considered on the plane, and α is the distance ratio, a scalar factor for controlling the distance.

Step 2, the query point is suppressed if it does not have the lowest fitting error and the smallest estimated angle within the directional neighborhood.

(2a), for the query point and the points within the directional neighborhood, their estimated angles are calculated from the parameters of their two half-planes. Their fitting errors are available from the two half-planes fitting process.

(2b), the query point is compared with points within the directional neighborhood. If the query point has both the lowest fitting error and the smallest estimated angle, it is preserved. Otherwise, the query point is suppressed.

Step 3, the number of preserved neighbors x_i is evaluated for each preserved point, and outlying false positives are removed.

(3a) a plane passing the preserved point and perpendicular to its edge direction is used to separate P_i^k into two subsets, $^{(1)}P_i^k$ and $^{(2)}P_i^k$.

(3b) the numbers of potential edge points from the two subsets $^{(1)}P_i^k$ and $^{(2)}P_i^k$ are counted, and the larger one is stored as the number of preserved neighbors x_i .

(3c) preserved points with too few neighbors will be removed by outlier removal.

ANMS is effective for reducing redundant detections in point clouds because it compares the geometrical quantities within a locally directional neighborhood. Edge points and non-edge points around edges cannot be previously differentiated since their geometrical quantities are often too close to effectively set a threshold in between. ANMS avoids this problem by making local comparisons. The comparisons are directionally constrained to avoid direct comparison among edge points and the suppression of them. The ANMS method is summarized in Algorithm 4-1.

Algorithm 4-1: Adapted-Non-Maximum-Suppression

Input: a point cloud with n points $P = \{\mathbf{p}_i \mid i = 1, 2, 3, \dots, n\}$;
 edge directions $D = \{\mathbf{d}_i \mid i = 1, 2, 3, \dots, n\}$, estimated by (4.1) ~ (4.4);
 point spacing $\{\mathbf{d}_{spacing,i} \mid i = 1, 2, 3, \dots, n\}$, estimated by (3.2);
 fitting errors $E = \{e_i \mid i = 1, 2, 3, \dots, n\}$;
 included angles of the two half-planes $\Theta = \{\theta_i \mid i = 1, 2, 3, \dots, n\}$;
 points selected by two half-planes detector P_{THPD} ;
 neighborhood size k ;
 distance ratio $\alpha = 0.75$;
 standard deviation multiplier $\beta = 2.0$.

Output: points detected by adapted non-maximum suppression P_{ANMS} .

- 1: Initialize $P_{preserved} \leftarrow \emptyset$
- 2: Initialize $P_{ANMS} \leftarrow \emptyset$
- 3: Initialize $X \leftarrow \emptyset$

```

4: for  $i \leftarrow 1$  to  $n$  do
5:   if  $\mathbf{p}_i \notin P_{THPD}$ 
6:     continue
7:   end if
8:   Initialize  $b_{preserve} \leftarrow \text{true}$  // indicates if  $\mathbf{p}_i$  should be preserved
9:   find neighborhood  $P_i^k$  from query point  $\mathbf{p}_i$  with kNN search [36], [37]
10:  for  $j \leftarrow 1$  to  $k$  do
11:    if  $\mathbf{p}_i^j \in P_{THPD}$  and  $\mathbf{p}_i^j \in \{\mathbf{p}_j \mid \mathbf{p}_j \in P_i^k \text{ and } \|(\mathbf{p}_i - \mathbf{p}_j) \cdot \mathbf{d}_i\| < \alpha \cdot d_{spacing,i}\}$ 
12:      if  $e_i^j < e_i$  or  $\theta_i^j < \theta_i$ 
13:         $b_{preserve} \leftarrow \text{false}$ 
14:        break
15:      end if
16:    end if
17:  end for
18:  if  $b_{preserve}$ 
19:     $P_{preserved} \leftarrow (P_{preserved} \cup \{\mathbf{p}_i\})$ 
20:  end if
21: end for
22: for each  $\mathbf{p}_i$  in  $P_{preserved}$  do // calculate the number of preserved neighbors
23:   find neighborhood  $P_i^k$  from query point  $\mathbf{p}_i$  with kNN search [36], [37]
24:   construct a plane passes  $\mathbf{p}_i$  and  $\perp$  to  $\mathbf{d}_i$ ,  $plane(\mathbf{p}_i, \mathbf{d}_i)$ 
25:   use  $plane(\mathbf{p}_i, \mathbf{d}_i)$  to divide the cartesian space,
      splitting  $P_i^k$  into 2 subsets,  $^{(1)}P_i^k$  and  $^{(2)}P_i^k$ .
26:    $^{(1)}x_i \leftarrow |^{(1)}P_i^k \cap P_{preserved}|$  // number of neighbors both in  $^{(1)}P_i^k$  and  $P_{ANMS}$ 
27:    $^{(2)}x_i \leftarrow |^{(2)}P_i^k \cap P_{preserved}|$  // number of neighbors both in  $^{(2)}P_i^k$  and  $P_{ANMS}$ 
28:    $x_i \leftarrow \max(^{(1)}x_i, ^{(2)}x_i)$ 
29:    $X \leftarrow (X \cup \{x_i\})$ 
30: end for
31:  $\mu_X \leftarrow \sum_{i=1}^n x_i$  // mean of  $X$ 
32:  $\sigma_X \leftarrow \sqrt{\sum_{i=1}^n (x_i - \mu_X)^2 / n}$  // standard deviation of  $X$ 
33: for each  $x_i$  in  $X$  do // outlier removal
34:   if  $x_i > (\mu_X - \beta \cdot \sigma_X)$ 
35:      $P_{ANMS} \leftarrow (P_{ANMS} \cup \{\mathbf{p}_i\})$ 
36:   end if
37: end for
38: return  $P_{ANMS}$ 

```

Chapter 5

5 Experiments and results

5.1 System overview

The system used in this work consists of a commercial structural light 3D scanner (Gocator 3210, LMI Technologies Inc.) mounted on a tripod and an object placed on a horizontal platform to be scanned, as shown in Figure 5-1. The 3D scanner has a field of view of $71.0\text{ mm} \times 98.0\text{ mm}$ at the near end (164.0 mm from the front of the scanner) and a field of view of $100.0\text{ mm} \times 154.0\text{ mm}$ at the far end (274.0 mm from the front of the scanner). The scanner has a projector that projects blue structured light onto the object, and the projected patterns are captured by its stereo camera and processed onboard to generate point clouds.

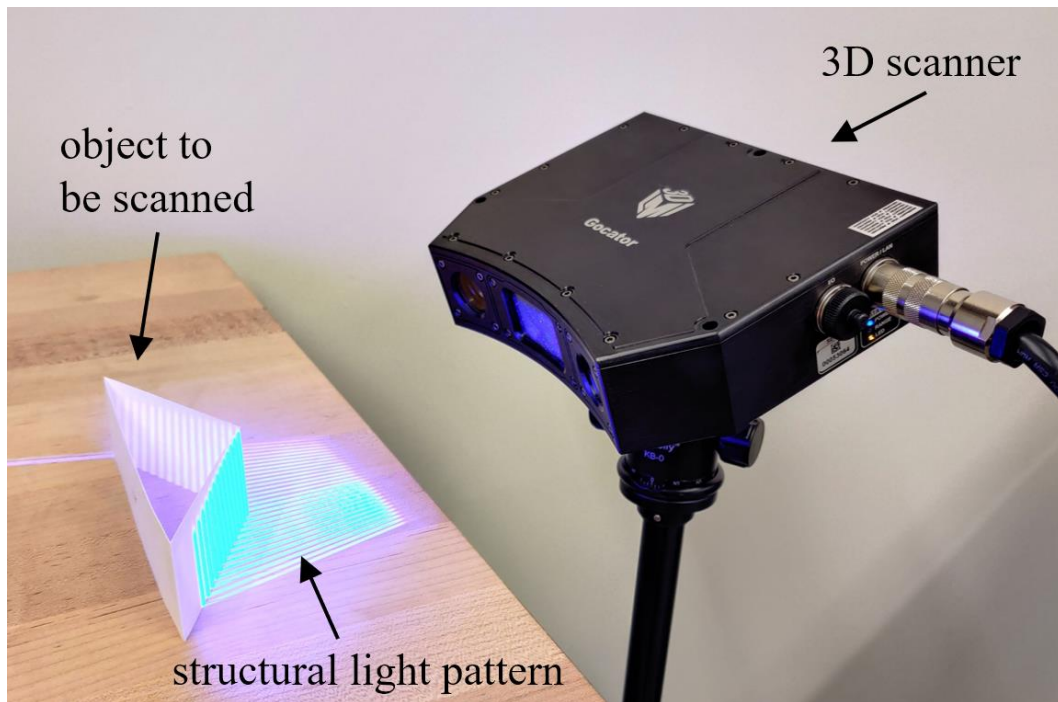


Figure 5-1. The system consists of a commercial structural light 3D scanner and the object to be scanned. The scanner projects structural light patterns onto the object and captures point clouds from the object.

The point clouds captured from the scanner are processed on a 64-bit Windows 10 PC (Intel i7-8550U processor with 16.0 GB of RAM). The point cloud processing programs are written and performed in Python (3.7.4) [68] with Open3D (0.10.0) [66], an open-source library for point cloud processing and NumPy (1.18.1) [69], [70], an open-source numerical computing library.

The steps for edge detection with the proposed geometry-based edge detector is presented in Figure 5-2. First, the point cloud is captured by the 3D scanner from the object. Second, we analyze neighborhoods of the point cloud and detect potential edge points with THP, enabled by the two half-planes fitting method. Parameters of the fitted half-planes, fitting error, and the indices of the potential edge points are passed down to the next step. Third, we refine the detection result with ANMS. This step produces the detected edge points for downstream applications, such as tool path generation.

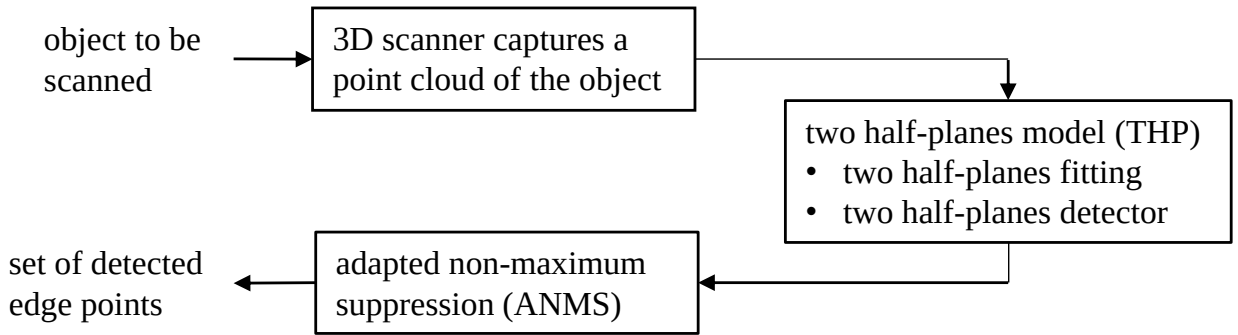


Figure 5-2. Edge detection process of the proposed geometry-based edge detector.

5.2 Performance of edge point detection using two half-planes model

To evaluate the proposed THP detector, we compare it with existing geometry-based detectors on how well missed detections are avoided under different edge angles.

We evaluate missed detections by calculating the recall value, as defined in (5.1) [71], with a range of 0.0 to 1.0. The recall value represents how well the detected edge points cover the ground truth edge points; i.e. how many ground truth edge points are detected. The higher the recall value, the better a detector is on avoiding missed detections. The lower the recall value, the more frequent missed detections occur. Note that although recall value evaluates the degree of missed detections, it does not entirely reflect the detection result since it does not consider false

positives (i.e. the case when non-edge points are marked as edge points). We evaluate performance regarding false positives in 5.3 and 5.4.

$$\begin{aligned}
 Recall &= \frac{\text{true positives}}{\text{true positives} + \text{false negatives}} \\
 &= \frac{\text{number of correctly detected edge points}}{\text{number of ground truth edge points}}
 \end{aligned} \tag{5.1}$$

The input point clouds are captured from objects with edge angles from 15° to 165° with an interval of 15° , made with 148 g/m^2 paper, as the object to be scanned shown in Figure 5-1. Each point cloud has around 5000 points with points the background manually removed. The ground truth edge points are carefully labeled in each point cloud.

The two half-planes detector (THP) is compared with the curvature-based detectors (surface variation detector [17] and mean shift detector [13]) and the normal-based detector (normal clustering detector [18]). Neighborhood size k is set to be 50 for the detectors for a balance between speed and performance. A lower neighborhood size $k = 10$ is used for the normal clustering detector since it has $O(k^6)$ time complexity for each neighborhood and demands prohibitively long computation time with larger k [18]. Thresholds for each detector are chosen to maximize the recall value for the point cloud with a 90° edge. When multiple thresholds achieve the maximum recall, we choose the threshold that has the least false positives.

Figure 5-3 shows the recall comparison of the geometry-based edge detectors in the point clouds with various edge angles. The qualitative comparison of detection results under selected edge angles (30° , 60° , and 90°) is presented in Figure 5-4.

All detectors have high recalls at 90° because their thresholds are tuned to obtain the highest recall value for the 90° input point cloud. The recall values of curvature-based and normal-based detectors are high around 90° , but quickly fall as the angle deviates from 90° . This is due to the geometrical quantities of these detectors (i.e. curvature and normal) are affected by edge angles. Consequently, since the threshold is only tuned for detecting the 90° edge, the threshold becomes undersized when applied on edges with a smaller angle (e.g. 30°) and become oversized when applied on edges with a larger angle (e.g. 150°). Edge points not detected by these detectors lead to a decrease in their recall value. The averaged recall across all tested angles is 0.40 for the

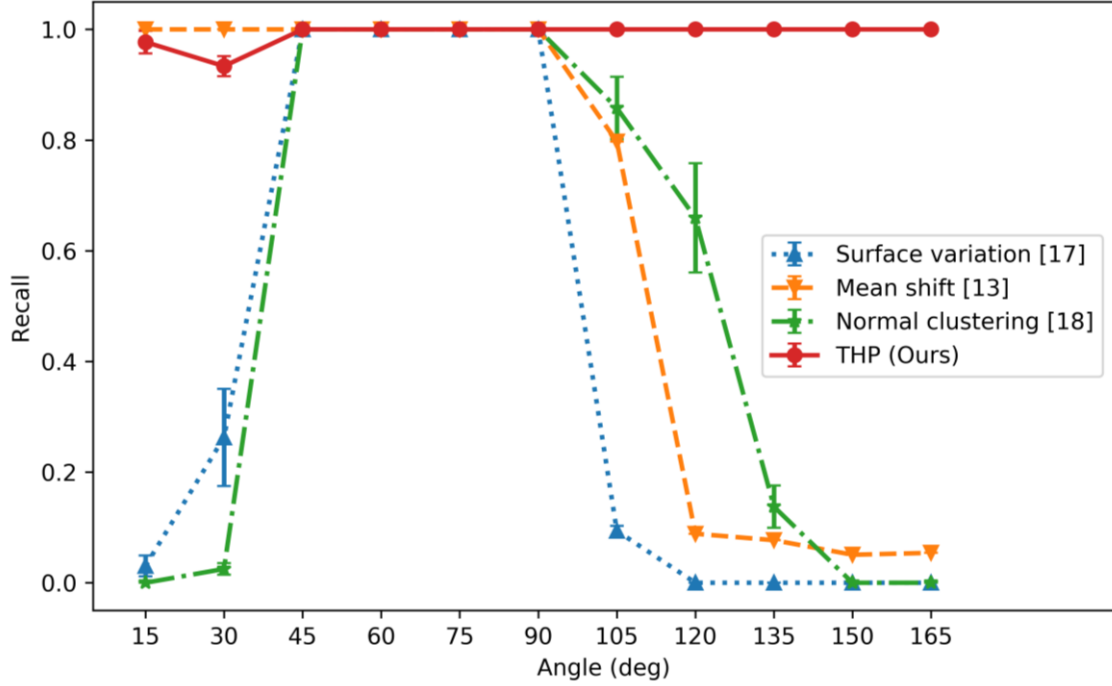


Figure 5-3. Recall value comparison of edge detectors under different edge angles. Error bars show standard deviation with 5 samples. Higher recall value means fewer missed detections.

surface variation detector, 0.64 for the mean shift detector, and 0.52 for the normal clustering detector. One exception is that the mean shift detector has high recall when the edge angle is close to 0° since the undersized threshold for the mean shift detector allows more points to be detected.

In comparison, the recall value of the proposed THP detector remains high across various edge angles (Figure 5-3, averaged at 0.99 across all tested edge angles). This is because the geometric quantity we used to differentiate edge points, i.e. the THP fitting error, is not dependent on the edge angle. Thus, the threshold tuned for the 90° input point cloud is also applicable for edges with other angles.

Note that in this experiment, we did not compare false positive detections (i.e. non-edge points detected as edge points). This is because false positives mostly occur near edges and can be processed with suppression methods. The mean shift detector has many false positives on the boundary of the point cloud because the mean shift detector does not differentiate edge points and boundary points. Although [13] claims it is developed for welding applications, it is more

desirable to only detect edge points because boundary points stem from point cloud incompleteness instead of the sampled object.

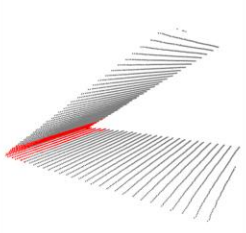
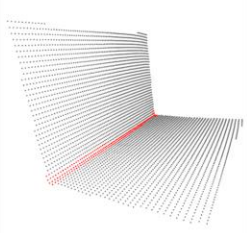
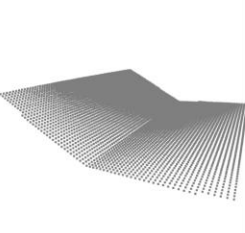
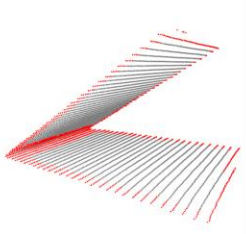
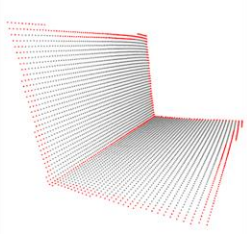
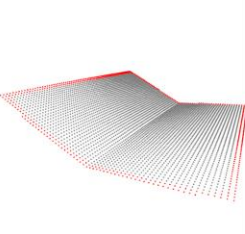
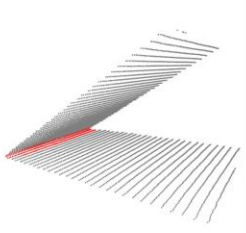
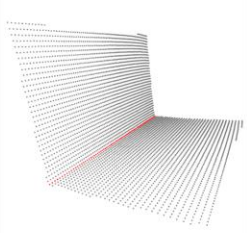
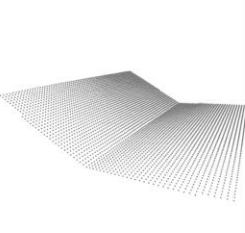
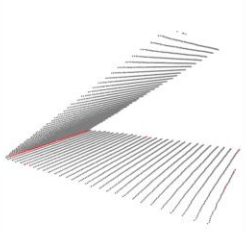
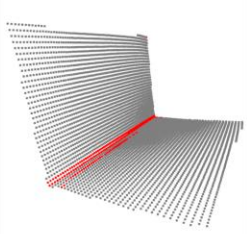
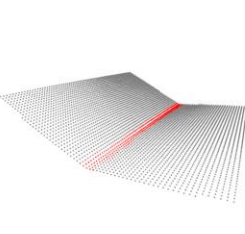
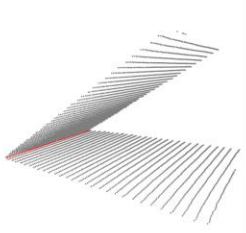
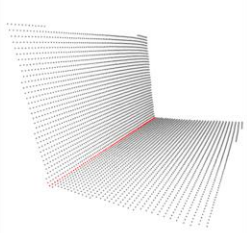
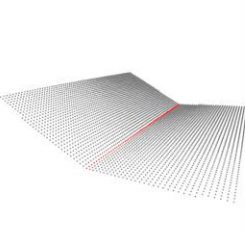
Detectors	edge angle		
	30°	90°	150°
Surface variation [17]			
Mean shift [13]			
Normal clustering [18]			
THP (Ours)			
Ground truth			

Figure 5-4. Qualitative comparison of detection results from the edge detectors. Red points and grey points represent detected edge points and non-edge points, respectively. Results with less missed detections when compared to the ground truth are better.

5.3 Performance of adapted non-maximum suppression

To evaluate the proposed adapted non-maximum suppression method, we compare the detection results before and after applying it on the potential edge points obtained from the two half-planes fitting process to investigate how well it reduces redundant detections.

We evaluate how well our method avoids redundant detections by calculating the F-score, as defined in (5.3) [71], with a range of 0.0 to 1.0. The F-score is a combination of the recall value and the precision value: the recall value given in (5.1) shows how well correctly detected edge points are preserved during the suppression. The precision value given in (5.2) represents how many detected edge points are correct [71], also with a range of 0.0 to 1.0. The higher the precision, the more detected points are correct and the fewer redundant detections. The F-score evaluates how well redundant detections are avoided while maintaining the recall obtained before suppression. It is calculated based on the comparison between the ground truth and the detection result after suppression is applied. The higher the F-score, the better the detection result matches the ground truth. The lower the F-score, the more redundant detections and/or missed detections are presented in the detection result.

$$\begin{aligned}
 Precision &= \frac{\text{true positives}}{\text{true positives} + \text{false positives}} \\
 &= \frac{\text{number of correctly detected edge points}}{\text{number of all detected edge points}}
 \end{aligned} \tag{5.2}$$

$$F\text{-score} = \frac{2}{\frac{1}{Recall} + \frac{1}{Precision}} \tag{5.3}$$

We used the same set of input point clouds and edge point ground truth as in 5.2. ANMS is compared with the detection result without any suppression, and the direct use of NMS without directional neighborhood. We did not compare our method with the thinning methods [61]–[63] since they introduce deviation and create new points instead of suppressing and preserving from the existing points. The suppression methods are applied for the two curvature-based detectors (the mean shift detector [13] and the surface variation detector [17]) and the proposed THP detector. Specifically, for value comparison within a neighborhood, we use the mean shift curvature (2.8) for the mean shift detector, surface variation curvature (2.7) for the surface

variation detector, and a combination of fitting error and edge angle for the THP detector (Algorithm 4-1). The normal clustering detector is excluded since it does not provide available quantities for local comparison, which is the foundation of NMS. ANMS uses the same neighborhood size as in 5.2. We use a small neighborhood size $k = 5$ for the direct NMS method since neighbors in any direction can be used to suppress existing detection and a large k results in suppression of most points.

Figure 5-5 shows the comparison of recall, precision, and F-score after the direct NMS or ANMS is applied on the edge detectors. Table 5-1 summarizes the average change of the metrics across all edge angles. The qualitative comparison of detection results with the suppression methods under the point cloud with the 90° edge is presented in Figure 5-6.

The direct NMS method can increase the precision value on average and under most edge angles, indicating a decline in redundant detections. This is because the geometrical quantities of redundant detections are not local maxima or local minima, and they can be suppressed by direct NMS.

However, the recall values are drastically decreased when direct NMS is applied. This is because direct NMS suppresses points that do not have the extreme value among their neighbors in all directions, including neighbors along the edge direction. When multiple edge points reside within the same neighborhood, only one of them can be preserved and the rest are suppressed, leading to missed detections and lower recall. Thus, the increase of precision is offset or even reverted by the decrease in the recall, giving overall lower F-scores.

In contrast, ANMS uses estimated edge direction to constrain neighbors, so that only neighbors reside on the plane perpendicular to the edge direction are compared with the query point. This avoids the comparison and suppression among correctly detected edge points. As a result, the decreases in recalls are kept minimum while the precisions are largely increased, giving higher F-scores. As summarized in Table 5-1, the F-score of the surface variation detector is increased from 0.19 to 0.24, for the mean shift detector from 0.11 to 0.32, and for the proposed THP from 0.67 to 0.99.

The results have also shown that the improvements are not exclusive to the proposed THP method. ANMS can be used to reduce redundant detections and improve the overall performance of existing edge detectors.

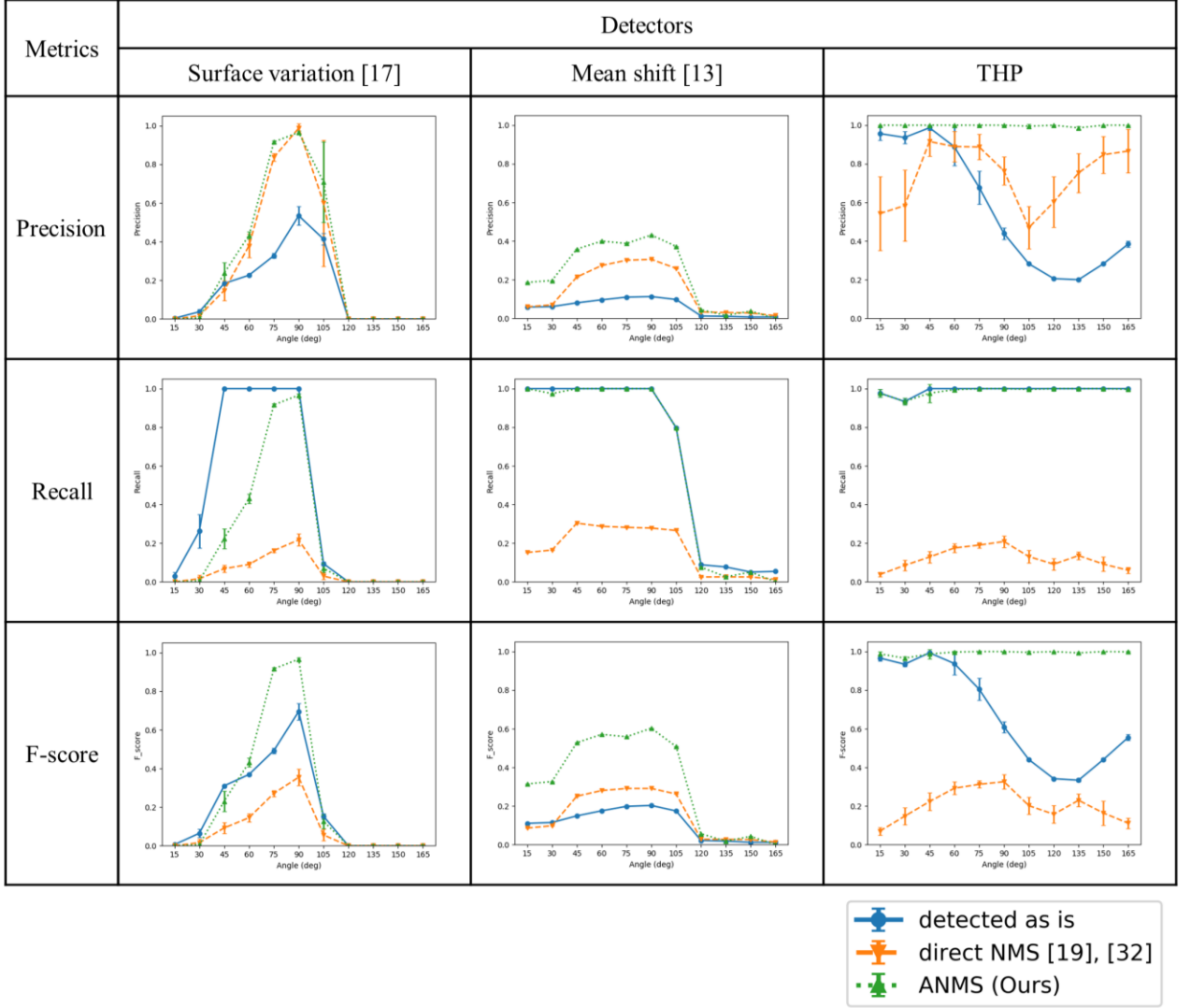


Figure 5-5. Precision, recall, and F-score comparison of edge detectors before and after applying direct non-maximum suppression (direct NMS) and adapted non-maximum suppression (ANMS). Higher precision means fewer redundant detections; higher recall means fewer missed detections; and higher F-score means better overall detection performance.

Table 5-1. Change of precision, recall, and F-score of the detectors after direct non-maximum suppression (direct NMS) and adapted non-maximum suppression (ANMS) are applied. Averaged across all tested edge angles. Higher precision means fewer redundant detections; higher recall means fewer missed detections; and higher F-score means better overall detection performance.

Detectors	Metrics	detected as is	direct NMS [19], [32]	ANMS (Ours)
Surface variation [17]	Precision	0.157	0.270	0.297
	Recall	0.399	0.053	0.238
	F-score	0.190	0.085	0.243
Mean shift [13]	Precision	0.060	0.145	0.221
	Recall	0.643	0.166	0.629
	F-score	0.109	0.151	0.321
THP	Precision	0.568	0.739	0.998
	Recall	0.992	0.121	0.989
	F-score	0.669	0.204	0.993

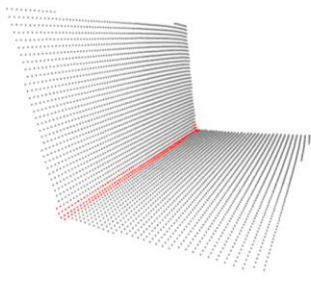
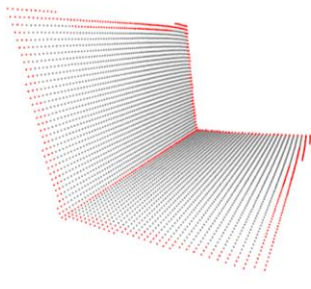
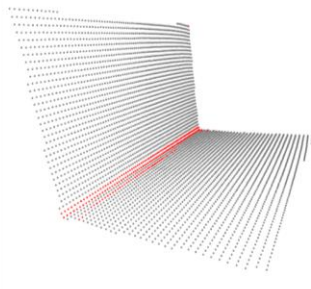
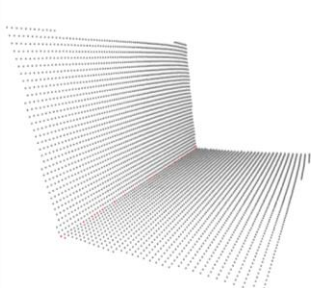
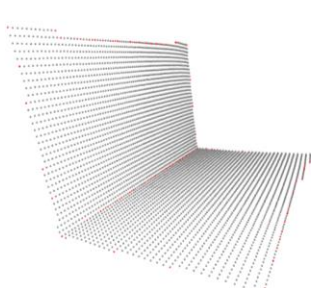
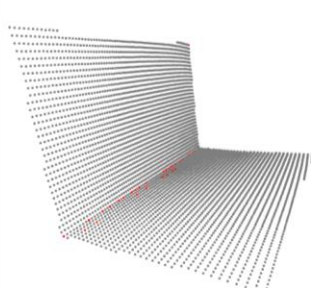
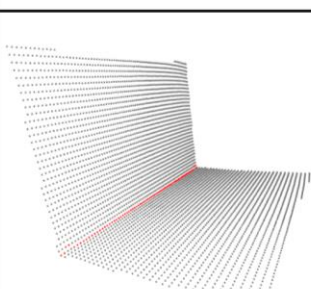
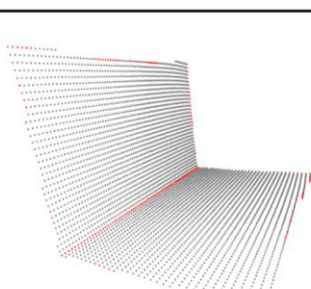
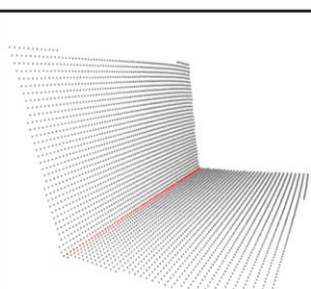
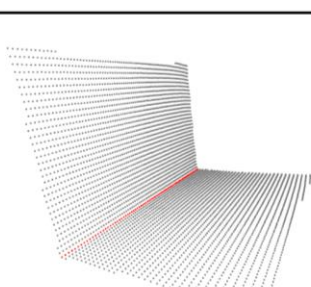
Suppression methods	Detectors		
	Surface variation [17]	Mean shift [13]	THP
Detected as is			
Direct NMS [19], [32]			
ANMS (Ours)			
Ground truth			

Figure 5-6. Qualitative comparison of detection results with direct non-maximum suppression (direct NMS) and adapted non-maximum suppression (ANMS) with the point cloud with a 90° edge. Red points and grey points represent detected edge points and non-edge points, respectively. The closer a detection result resembles the ground truth, the better it is.

5.4 Overall performance evaluation

To evaluate the overall performance of our detector with the two proposed techniques under industrial settings, we compare our detector (THP+ANMS) with existing geometry-based detectors on detection performance with point clouds scanned from industrial objects. Similar to 5.3, we evaluate the performance of the detectors in F-scores. We also compare the speed of each detector in terms of the number of processed points per second.

We used point clouds scanned from industrial objects with the 3D scanner. The objects include a steel flywheel, an aluminum right angle connector, and a plastic trigger sprayer head as shown in Figure 5-7. The ground truth edge points of each point cloud are carefully labeled. The three point clouds have different angles for their edges: the flywheel point cloud captures the edge of a cylinder and is consist of edge about 90° , the connector point cloud captures the intersection curve between a 45° inclined plane and a horizontal cylinder and has an edge with an angle ranging from about 90° to 135° , the sprayer point cloud captures the intersection curve of two complex surfaces and has varying blunt edge angles.

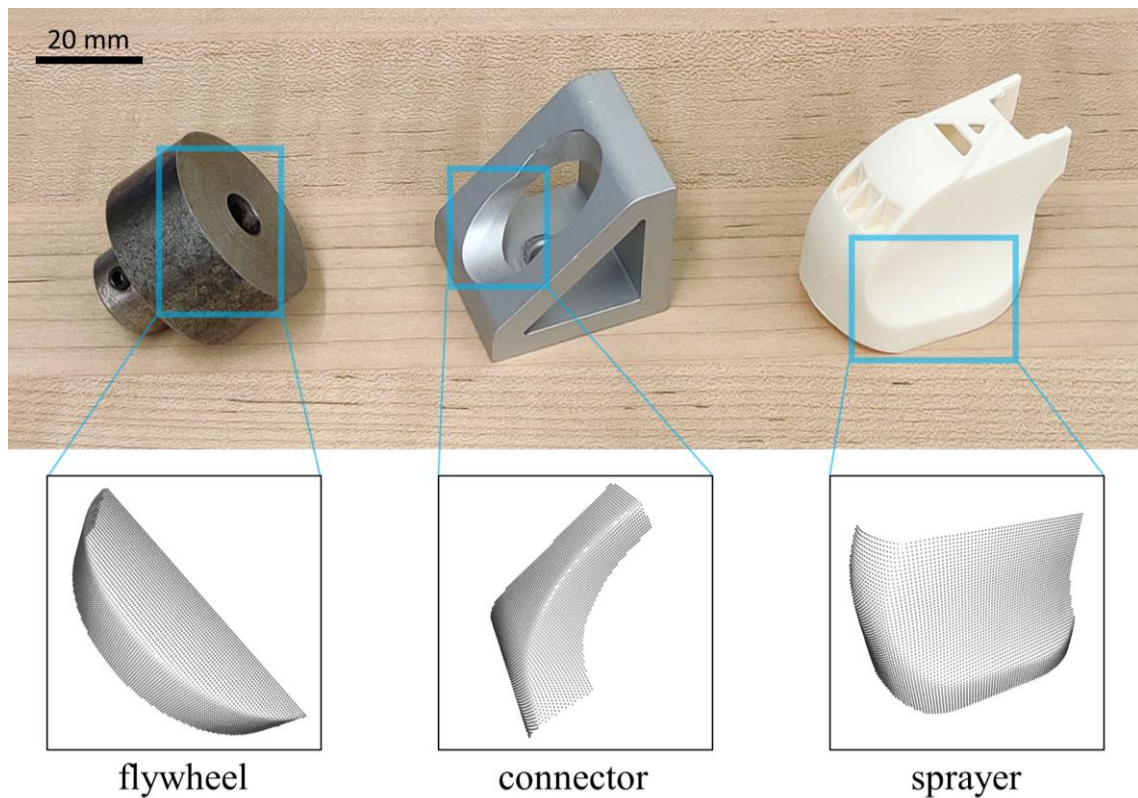


Figure 5-7. The industrial objects and their point clouds captured from the 3D scanner.

We compare the detection results of our proposed detector (THP+ANMS), the curvature-based detectors (the mean shift detector [13] and the surface variation detector [17]), and the normal-based detector [18]. For our proposed detector, we use the same parameters as in 5.2 and 5.3, since the parameters do not need to be adjusted for edges with different angles. For the existing detectors, we use two sets of thresholds: the fixed thresholds that were used in 5.2 and 5.3, and manually tuned thresholds for each point cloud. The thresholds used for each edge detector are summarized in Table 5-2. Figure 5-8 shows the comparison of F-scores achieved by the edge detectors. The qualitative comparison of the detection result is summarized in Figure 5-9. The overall performance of the detectors is summarized in Table 5-3.

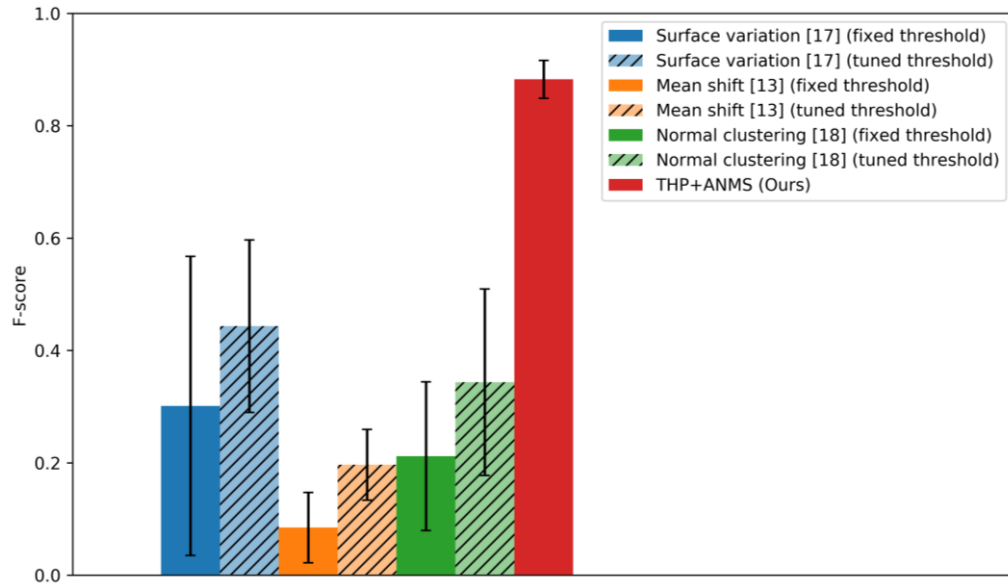


Figure 5-8. F-score comparison of edge detectors applied on point clouds captured from industrial objects. Values averaged across the three point clouds. Error bars show standard deviation.

Table 5-2. Thresholds used for the edge detectors. For the existing detectors, both fixed and tuned thresholds are used.

	Surface variation [17]	Mean shift [13]	Normal clustering [18]	THP+ANMS (Ours)
fixed threshold	0.07	1.1	1.0	
tuned threshold (flywheel)	0.07	1.0	1.1	0.5 (no need to tune)
tuned threshold (connector)	0.04	0.7	1.2	
tuned threshold (sprayer)	0.02	0.5	1.3	

The proposed THP+ANMS detector receives 0.88 F-score for all three point clouds captured from industrial point clouds without the need to tune thresholds. In comparison, the F-scores of the existing detectors are relatively lower (0.08~0.30) with fixed thresholds, since fixed thresholds are no longer suitable for the edge angles in the tested point clouds. The F-scores increased for the existing detectors when their thresholds are manually tuned for each point cloud, but because the edge angles within each point cloud are varying, their increased F-scores are still lower (0.20~0.44) than that of the proposed detector. The surface variation detector and mean shift detector can process thousands of points per second since little computation (PCA and average coordinate) is involved for the neighborhood of each point. The proposed detector

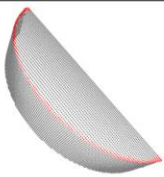
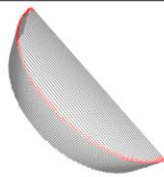
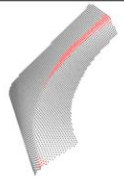
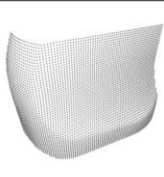
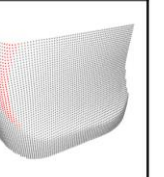
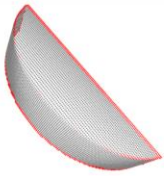
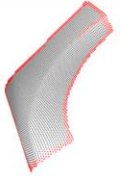
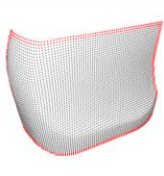
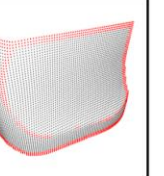
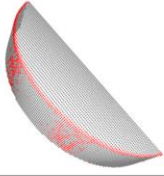
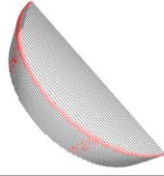
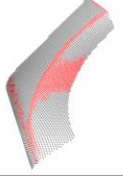
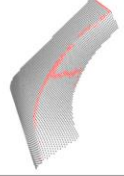
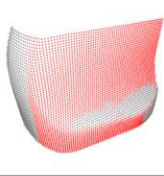
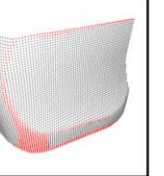
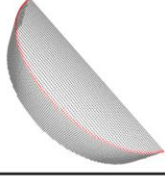
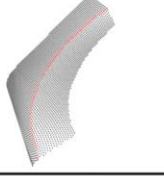
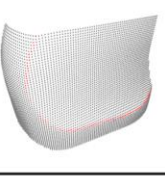
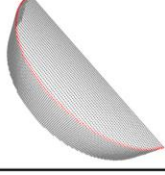
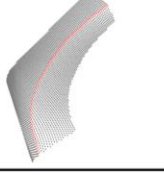
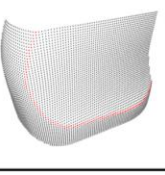
Detectors	Flywheel		Connector		Sprayer	
	Fixed threshold	Tuned threshold	Fixed threshold	Tuned threshold	Fixed threshold	Tuned threshold
Surface variation [17]						
Mean shift [13]						
Normal clustering [18]						
THP+ANMS (Ours)						
Ground truth						

Figure 5-9. Qualitative comparison of detection results of the edge detectors applied on point clouds captured from industrial objects. Red points and grey points represent detected edge points and non-edge points, respectively.

processes 30.4 points per second since THP requires multiple iterations for the Monte Carlo method. The normal clustering detector has a lower processing speed compared to our detector because the HAC clustering process requires intensive calculation.

Table 5-3. Overall performance of the edge detectors. Values show the means and standard deviations. Values enclosed in parenthesis are associated with fixed thresholds.

	Surface variation [17]	Mean shift [13]	Normal clustering [18]	THP+ANMS (Ours)
F-scores	0.44 ± 0.15 (0.30 ± 0.27)	0.19 ± 0.06 (0.08 ± 0.06)	0.34 ± 0.17 (0.21 ± 0.13)	0.88 ± 0.03
thresholds used	0.02, 0.04, 0.07 (0.07)	0.5, 0.7, 1.0 (1.1)	1.1, 1.2, 1.3 (1.0)	0.5
points per second	8699.6 ± 175.6	24331.0 ± 696.4	5.4 ± 0.1	30.4 ± 0.8

Chapter 6

6 Conclusions and future work

6.1 Conclusions

Edge detection in 3D point clouds is crucial for point cloud processing, but existing detectors suffer from missed detections and redundant detections and require thresholds to be tuned for every point clouds.

This work proposed a new two-stage geometry-based detector to solve incorrect detection. First, to reduce missed detections, we eliminate the dependency of thresholds on edge angles. Different from traditional geometry-based methods that rely on angle-dependent geometrical quantities to detect edges, we fit the neighborhoods with a novel two half-planes model (THP), and the fitting errors are not affected by edge angles. Thus, we can use a fixed threshold to detect edge points with a wide range of edge angles.

Second, to reduce redundant detections, we adapt non-maximum suppression (NMS) for point clouds, so that duplicate detection around edges can be reduced. Traditionally, NMS is designed for 2D grid-like images and relies on the magnitudes and directions of intensity gradients. However, point clouds in general do not contain intensity information. The adapted non-maximum suppression (ANMS) method uses estimated edge directions to provide directional constraints for the neighborhoods and conduct local comparisons based on fitting errors and estimated angles. These quantities used by ANMS do not rely on intensities and can be derived from point coordinates stored within point clouds. By applying ANMS, redundant detections can be suppressed while preserving correctly detected points.

Experiment results show that THP achieved 0.99 recall for various edge angles (vs 0.40~0.64 for existing detectors), ANMS increased the F-score from 0.67 to 0.99, and the THP+ANMS detector received 0.88 F-score for point clouds captured from industrial objects without the need to tune thresholds (vs 0.20~0.44 for existing detectors with tuned thresholds).

6.2 Future work

The following are examples of projects and experiments that can be undertaken for future research:

1. The proposed edge detector uses THP to model point cloud neighborhoods. Although two half-planes can be used to represent both edges and planes, geometrical features generated from more than two planes, such as corners, cannot be properly represented. To avoid this issue, models involving multiple faces (or half-planes) can be used to fit point cloud neighborhoods. The number of faces can be dynamically determined so that various features including planes, edges, and corners can be contained with the same model.
2. The processing speed of the proposed detector prevents it from being applied in real-time applications involving large point cloud sizes. The speed can be improved by taking advantage of faster computation hardware, enabling parallel programming, using compiled programming languages, and using faster libraries. The processing speed can also be improved by reducing the number of iterations (with a lower desired probability of successfully finding a model) or neighborhood size.
3. The proposed edge detector analyzes the neighborhood of a query point with a fixed neighborhood size. This neighborhood size can be changed by the user according to application types and sampling density of the point cloud. However, it can be desirable to detect features across various scales. A multi-scale approach with a wide range of neighborhood sizes can be used to obtain smaller and larger neighborhoods. This will allow geometric quantities to be calculated at different scales. The geometric quantities can be merged according to their saliency and used to collectively explain the query point.

References

- [1] R. B. Rusu, Z. C. Marton, N. Blodow, M. Dolha, and M. Beetz, “Towards 3D Point cloud based object maps for household environments,” *Robot. Auton. Syst.*, vol. 56, no. 11, pp. 927–941, Nov. 2008, doi: 10.1016/j.robot.2008.08.005.
- [2] J. Geng, “Structured-light 3D surface imaging: a tutorial,” *Adv. Opt. Photonics*, vol. 3, no. 2, pp. 128–160, Jun. 2011, doi: 10.1364/AOP.3.000128.
- [3] E. Che, J. Jung, and M. J. Olsen, “Object Recognition, Segmentation, and Classification of Mobile Laser Scanning Point Clouds: A State of the Art Review,” *Sensors*, vol. 19, no. 4, Art. no. 4, Jan. 2019, doi: 10.3390/s19040810.
- [4] A. Kolb, E. Barth, R. Koch, and R. Larsen, “Time-of-Flight Cameras in Computer Graphics,” *Comput. Graph. Forum*, vol. 29, no. 1, pp. 141–159, 2010, doi: 10.1111/j.1467-8659.2009.01583.x.
- [5] W. Li, Q. Guo, M. K. Jakubowski, and M. Kelly, “A New Method for Segmenting Individual Trees from the Lidar Point Cloud,” *Photogramm. Eng. Remote Sens.*, vol. 78, no. 1, pp. 75–84, Jan. 2012, doi: 10.14358/PERS.78.1.75.
- [6] Z. M. Bi and B. Kang, “Sensing and responding to the changes of geometric surfaces in flexible manufacturing and assembly,” *Enterp. Inf. Syst.*, vol. 8, no. 2, pp. 225–245, Mar. 2014, doi: 10.1080/17517575.2012.654826.
- [7] J. Wang *et al.*, “Integrating BIM and LiDAR for Real-Time Construction Quality Control,” *J. Intell. Robot. Syst.*, vol. 79, no. 3, pp. 417–432, Aug. 2015, doi: 10.1007/s10846-014-0116-8.
- [8] B. Y. L. Li, A. S. Mian, W. Liu, and A. Krishna, “Using Kinect for face recognition under varying poses, expressions, illumination and disguise,” in *2013 IEEE Workshop on Applications of Computer Vision (WACV)*, Jan. 2013, pp. 186–192, doi: 10.1109/WACV.2013.6475017.

- [9] G. Farin, J. Hoschek, and M.-S. Kim, *Handbook of Computer Aided Geometric Design*. Elsevier, 2002.
- [10] S. A. N. Gilani, M. Awrangjeb, and G. Lu, “Segmentation of Airborne Point Cloud Data for Automatic Building Roof Extraction,” *GIScience Remote Sens.*, vol. 55, no. 1, pp. 63–89, Jan. 2018, doi: 10.1080/15481603.2017.1361509.
- [11] H. Huang, S. Wu, M. Gong, D. Cohen-Or, U. Ascher, and H. (Richard) Zhang, “Edge-aware point set resampling,” *ACM Trans. Graph.*, vol. 32, no. 1, pp. 1–12, Jan. 2013, doi: 10.1145/2421636.2421645.
- [12] N. Salman, M. Yvinec, and Q. Merigot, “Feature Preserving Mesh Generation from 3D Point Clouds,” *Comput. Graph. Forum*, vol. 29, no. 5, pp. 1623–1632, Sep. 2010, doi: 10.1111/j.1467-8659.2010.01771.x.
- [13] S. M. Ahmed, Y. Z. Tan, C. M. Chew, A. A. Mamun, and F. S. Wong, “Edge and Corner Detection for Unorganized 3D Point Clouds with Application to Robotic Welding,” in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Madrid, Oct. 2018, pp. 7350–7355, doi: 10.1109/IROS.2018.8593910.
- [14] Changhyun Choi, A. J. B. Trevor, and H. I. Christensen, “RGB-D edge detection and edge-based registration,” in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Tokyo, Nov. 2013, pp. 1568–1575, doi: 10.1109/IROS.2013.6696558.
- [15] S. Li and D. Lee, “RGB-D SLAM in Dynamic Environments Using Static Point Weighting,” *IEEE Robot. Autom. Lett.*, vol. 2, no. 4, pp. 2263–2270, Oct. 2017, doi: 10.1109/LRA.2017.2724759.
- [16] D. Panozzo, E. Puppo, and L. Rocca, “Efficient multi-scale curvature and crease estimation,” *Proc. Comput. Graph. Comput. Vis. Math. Brno Czech Repubic*, vol. 1, no. 6, 2010.
- [17] D. Bazazian, J. R. Casas, and J. Ruiz-Hidalgo, “Fast and Robust Edge Extraction in Unorganized Point Clouds,” in *2015 International Conference on Digital Image*

- Computing: Techniques and Applications (DICTA)*, Adelaide, Australia, Nov. 2015, pp. 1–8, doi: 10.1109/DICTA.2015.7371262.
- [18] C. Weber, S. Hahmann, and H. Hagen, “Methods for Feature Detection in Point Clouds,” p. 10 pages, 2011, doi: 10.4230/OASICS.VLUDS.2010.90.
- [19] J. Canny, “A Computational Approach to Edge Detection,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986, doi: 10.1109/TPAMI.1986.4767851.
- [20] L. Shao, J. Han, P. Kohli, and Z. Zhang, *Computer vision and machine learning with RGB-D sensors*, vol. 20. Springer, 2014.
- [21] M. Berger *et al.*, “A Survey of Surface Reconstruction from Point Clouds,” *Comput. Graph. Forum*, vol. 36, no. 1, pp. 301–329, 2017, doi: 10.1111/cgf.12802.
- [22] G. Lee, J. Mou, and Y. Shen, “Sampling strategy design for dimensional measurement of geometric features using coordinate measuring machine,” *Int. J. Mach. Tools Manuf.*, vol. 37, no. 7, pp. 917–934, Jul. 1997, doi: 10.1016/S0890-6955(96)00096-X.
- [23] S. May, B. Werner, H. Surmann, and K. Pervolz, “3D time-of-flight cameras for mobile robotics,” in *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Oct. 2006, pp. 790–795, doi: 10.1109/IROS.2006.281670.
- [24] X. Yue, B. Wu, S. A. Seshia, K. Keutzer, and A. L. Sangiovanni-Vincentelli, “A LiDAR Point Cloud Generator: from a Virtual World to Autonomous Driving,” in *Proceedings of the 2018 ACM on International Conference on Multimedia Retrieval*, New York, NY, USA, Jun. 2018, pp. 458–464, doi: 10.1145/3206025.3206080.
- [25] A. S. Glassner, *An Introduction to Ray Tracing*. Elsevier, 1989.
- [26] G. Turk and M. Levoy, “Zippered polygon meshes from range images,” in *Proceedings of the 21st annual conference on Computer graphics and interactive techniques*, New York, NY, USA, Jul. 1994, pp. 311–318, doi: 10.1145/192161.192241.

- [27] D. Belton and D. D. Lichti, "Classification and segmentation of terrestrial laser scanner point clouds using local variance information," *Int Arch Photogramm Remote Sens Spat Inf Sci*, vol. 36, no. 5, pp. 44–49, 2006.
- [28] H. S. Kim, H. K. Choi, and K. H. Lee, "Feature detection of triangular meshes based on tensor voting theory," *Comput.-Aided Des.*, vol. 41, no. 1, pp. 47–58, Jan. 2009, doi: 10.1016/j.cad.2008.12.003.
- [29] A. D. Sappa and M. Devy, "Fast range image segmentation by an edge detection strategy," in *Proceedings Third International Conference on 3-D Digital Imaging and Modeling*, May 2001, pp. 292–299, doi: 10.1109/IM.2001.924460.
- [30] Y. Sun, D. L. Page, J. K. Paik, A. Koschan, and M. A. Abidi, "Triangle mesh-based edge detection and its application to surface segmentation and adaptive surface smoothing," in *Proceedings. International Conference on Image Processing*, Sep. 2002, vol. 3, pp. 825–828 vol.3, doi: 10.1109/ICIP.2002.1039099.
- [31] F. P. Preparata and M. I. Shamos, *Computational Geometry: An Introduction*. Springer Science & Business Media, 2012.
- [32] R. Szeliski, *Computer vision: algorithms and applications*. London ; New York: Springer, 2011.
- [33] O. Monga, R. Deriche, and J.-M. Rocchisani, "3D edge detection using recursive filtering: Application to scanner images," *CVGIP Image Underst.*, vol. 53, no. 1, pp. 76–87, Jan. 1991, doi: 10.1016/1049-9660(91)90006-B.
- [34] D. Shin, C. C. Fowlkes, and D. Hoiem, "Pixels, Voxels, and Views: A Study of Shape Representations for Single View 3D Object Shape Prediction," 2018, pp. 3061–3069, Accessed: Aug. 11, 2020. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Shin_Pixels_Voxels_and_CVPR_2018_paper.html.
- [35] S. Song, F. Yu, A. Zeng, A. X. Chang, M. Savva, and T. Funkhouser, "Semantic Scene Completion From a Single Depth Image," 2017, pp. 1746–1754, Accessed: Aug. 11, 2020.

[Online]. Available:

https://openaccess.thecvf.com/content_cvpr_2017/html/Song_Semantic_Scene_Completion_CVPR_2017_paper.html.

- [36] M. Muja and D. G. Lowe, “Fast approximate nearest neighbors with automatic algorithm configuration,” in *Proceedings of the Fourth International Conference on Computer Vision Theory and Applications*, Lisboa, Portugal, 2009, pp. 331–340, doi: 10.5220/0001787803310340.
- [37] J. H. Friedman, J. L. Bentley, and R. A. Finkel, “An algorithm for finding best matches in logarithmic expected time,” *ACM Trans. Math. Softw. TOMS*, vol. 3, no. 3, pp. 209–226, 1977.
- [38] A. Nurunnabi, G. West, and D. Belton, “Outlier detection and robust normal-curvature estimation in mobile laser scanning 3D point cloud data,” *Pattern Recognit.*, vol. 48, no. 4, pp. 1404–1419, Apr. 2015, doi: 10.1016/j.patcog.2014.10.014.
- [39] B. Li, R. Schnabel, R. Klein, Z. Cheng, G. Dang, and S. Jin, “Robust normal estimation for point clouds with sharp features,” *Comput. Graph.*, vol. 34, no. 2, pp. 94–106, Apr. 2010, doi: 10.1016/j.cag.2010.01.004.
- [40] T. K. Dey, G. Li, and J. Sun, “Normal estimation for point clouds: a comparison study for a Voronoi based method,” in *Proceedings Eurographics/IEEE VGTC Symposium Point-Based Graphics, 2005.*, Jun. 2005, pp. 39–46, doi: 10.1109/PBG.2005.194062.
- [41] M. Pauly, M. Gross, and L. P. Kobbelt, “Efficient simplification of point-sampled surfaces,” in *IEEE Visualization, 2002. VIS 2002.*, Boston, MA, USA, 2002, pp. 163–170, doi: 10.1109/VISUAL.2002.1183771.
- [42] J. Behley, V. Steinhage, and A. B. Cremers, “Efficient radius neighbor search in three-dimensional point clouds,” in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, May 2015, pp. 3625–3630, doi: 10.1109/ICRA.2015.7139702.
- [43] J. L. Bentley, “A survey of techniques for fixed radius near neighbor searching,” Stanford University, Stanford, CA, USA, Technical Report, 1975.

- [44] K. F. Mulchrone, “Application of Delaunay triangulation to the nearest neighbour method of strain analysis,” *J. Struct. Geol.*, vol. 25, no. 5, pp. 689–702, May 2003, doi: 10.1016/S0191-8141(02)00067-6.
- [45] I. T. Jolliffe and J. Cadima, “Principal component analysis: a review and recent developments,” *Philos. Trans. R. Soc. Math. Phys. Eng. Sci.*, vol. 374, no. 2065, p. 20150202, Apr. 2016, doi: 10.1098/rsta.2015.0202.
- [46] H. Hoppe, T. DeRose, T. Duchamp, J. McDonald, and W. Stuetzle, “Surface reconstruction from unorganized points,” in *Proceedings of the 19th annual conference on Computer graphics and interactive techniques*, New York, NY, USA, Jul. 1992, pp. 71–78, doi: 10.1145/133994.134011.
- [47] K. Pearson, “LIII. On lines and planes of closest fit to systems of points in space,” Nov. 1901, doi: 10.1080/14786440109462720.
- [48] C. M. Shakarji, “Least-Squares Fitting Algorithms of the NIST Algorithm Testing System,” *J. Res. Natl. Inst. Stand. Technol.*, vol. 103, no. 6, pp. 633–641, 1998, doi: 10.6028/jres.103.043.
- [49] L. Wasserman, *All of Nonparametric Statistics*. New York: Springer-Verlag, 2006.
- [50] H. K. Heidemann, “Lidar base specification,” U.S. Geological Survey, Reston, VA, USGS Numbered Series 11-B4, 2012. doi: 10.3133/tm11B4.
- [51] P. Petersen, *Riemannian Geometry*, vol. 171. Cham: Springer International Publishing, 2016.
- [52] R. Schnabel, R. Wahl, and R. Klein, “Efficient RANSAC for Point-Cloud Shape Detection,” *Comput. Graph. Forum*, vol. 26, no. 2, pp. 214–226, Jun. 2007, doi: 10.1111/j.1467-8659.2007.01016.x.
- [53] M. A. Fischler and R. C. Bolles, “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.

- [54] F. Tarsha-Kurdi, T. Landes, and P. Grussenmeyer, “Hough-Transform and Extended RANSAC Algorithms for Automatic Detection of 3D Building Roof Planes from Lidar Data,” p. 7, 2007.
- [55] D. Borrmann, J. Elseberg, K. Lingemann, and A. Nüchter, “The 3D Hough Transform for plane detection in point clouds: A review and a new accumulator design,” *3D Res.*, vol. 2, no. 2, p. 3, Nov. 2011, doi: 10.1007/3DRes.02(2011)3.
- [56] R. Hulik, M. Spanel, P. Smrz, and Z. Materna, “Continuous plane detection in point-cloud data based on 3D Hough Transform,” *J. Vis. Commun. Image Represent.*, vol. 25, no. 1, pp. 86–97, Jan. 2014, doi: 10.1016/j.jvcir.2013.04.001.
- [57] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition*. Springer Science & Business Media, 2009.
- [58] O. Maimon and L. Rokach, Eds., *Data Mining and Knowledge Discovery Handbook*. Boston, MA: Springer US, 2010.
- [59] D. P. Kroese, T. Brereton, T. Taimre, and Z. Botev, “Why the Monte Carlo method is so important today,” 2014, doi: 10.1002/WICS.1314.
- [60] P. J. Rousseeuw and A. M. Leroy, *Robust Regression and Outlier Detection*. John Wiley & Sons, 2005.
- [61] J. Nie, “Extracting feature lines from point clouds based on smooth shrink and iterative thinning,” *Graph. Models*, vol. 84, pp. 38–49, Mar. 2016, doi: 10.1016/j.gmod.2016.04.001.
- [62] H. Lin, W. Chen, and G. Wang, “Curve reconstruction based on an interval B-spline curve,” *Vis. Comput.*, vol. 21, no. 6, pp. 418–427, Jul. 2005, doi: 10.1007/s00371-005-0304-4.
- [63] I.-K. Lee, “Curve reconstruction from unorganized points,” *Comput. Aided Geom. Des.*, vol. 17, no. 2, pp. 161–177, Feb. 2000, doi: 10.1016/S0167-8396(99)00044-8.

- [64] S. Qiu, G. Wen, Z. Deng, J. Liu, and Y. Fan, “Accurate non-maximum suppression for object detection in high-resolution remote sensing images,” *Remote Sens. Lett.*, vol. 9, no. 3, pp. 237–246, Mar. 2018, doi: 10.1080/2150704X.2017.1415473.
- [65] A. Neubeck and L. Van Gool, “Efficient Non-Maximum Suppression,” in *18th International Conference on Pattern Recognition (ICPR’06)*, Aug. 2006, vol. 3, pp. 850–855, doi: 10.1109/ICPR.2006.479.
- [66] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A Modern Library for 3D Data Processing,” *ArXiv180109847 Cs*, Jan. 2018, Accessed: Aug. 10, 2020. [Online]. Available: <http://arxiv.org/abs/1801.09847>.
- [67] R. B. Rusu, *Semantic 3D Object Maps for Everyday Robot Manipulation*, vol. 85. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013.
- [68] G. van Rossum and F. L. Drake, *The Python Language Reference Manual*. Network Theory Ltd., 2011.
- [69] S. van der Walt, S. C. Colbert, and G. Varoquaux, “The NumPy Array: A Structure for Efficient Numerical Computation,” *Comput. Sci. Eng.*, vol. 13, no. 2, pp. 22–30, Mar. 2011, doi: 10.1109/MCSE.2011.37.
- [70] T. E. Oliphant, *A guide to NumPy*, vol. 1. Trelgol Publishing USA, 2006.
- [71] D. M. Powers, “Evaluation: from Precision, Recall and F-measure to ROC, Informedness, Markedness and Correlation,” *J. Mach. Learn. Technol.*, vol. 2, no. 1, pp. 37–63, Dec. 2011.